

在内核管理结构和算法讨论完毕后，我们继续讨论了用户空间进程内存管理。进程内存管理不同于内核内存管理。我们讨论了进程内存布局和内存中各进程部分如何分配和映射。依照进程内存管理流程，我们介绍了缺页的概念，这个中断处理程序负责管理内存所缺少的页面。

4.12 项目：进程内存映射

现在看看我们自己程序的内存是什么样子的。该项目包括写一个用户空间程序，利用它来描述进程内存的分布。就这个项目而言，我们要创建一个简单的共享库和一个利用其函数的用户空间程序。在该程序中，我们会打印一些变量的位置，并且对比打印的位置和进程内存映射来确定变量和函数被存放的位置。

第一步我们先来创建共享库。共享库可以只有一个函数，该函数将从主程序调用。我们希望打印该函数中局部变量的地址。共享库看起来像下面这样：

```
-----
lkpsinglefoo.c
mylibfoo()
{
    int libvar;
    printf("variable libvar \t location: 0x%x\n", &libvar);
}
-----
```

编译 singlefoo.c 并将其链接成共享库：

```
#lkp>gcc -c lkpsinglefoo.c
#lkp>gcc lkpsinglefoo.o -o liblkpsinglefoo.so -shared -lc
```

-shared 和 -lc 标记是链接选项。-shared 选项要求产生可和其他对象链接在一起的共享对象，-lc 标记表示链接时需要搜索 C 库。

上述命令产生一个名为 liblkpsinglefoo.so 的文件，使用它之前需要将其复制到 /lib 目录下。

下面是我们将调用的链接在共享库中的主应用程序。

```
-----
lkpmem.c
#include <fcntl.h>

int globalvar1;
int globalvar2 = 3;

void mylocalfoo()
{
    int functionvar;
    printf("variable functionvar \t location: 0x%x\n", &functionvar);
}

int main()
{
    void *localvar1 = (void *)malloc(2048)
    printf("variable globalvar1 \t location: 0x%x\n", &globalvar1);
    printf("variable globalvar2 \t location: 0x%x\n", &globalvar2);
}
```

```

printf("variable localvar1 \t location: 0x%x\n", &localvar1);

mylibfoo();
mylocalfoo();

while(1);
return(0);
}
-----

```

按照如下命令编译 lkpmem.c:

```
#lkp>gcc -o lkpmem lkpmem.c -llkplibsingelfoo
```

执行命令 lkpmem 时,系统将打印出各种变量的内存位置。while(1); 内的函数块不被返回,以便你来得及获得进程 PID,并查看进程的内存映射。为此,要执行如下命令:

```

#lkp>./lkpmem
#lkp>ps aux | grep lkpmem
#lkp>cat /proc/<pid>/maps

```

通过上述命令你可看到每个变量所在的内存段。

4.13 习题

1. 为什么不能从一个普通的可执行文件执行进程或者为什么程序不能共享内存的数据段?
2. 下列函数在 3 次嵌套调用后,栈将如何变化?

```

foo(){
    int a;
    foo()
}

```

如果继续执行下去,会有什么问题发生?

3. 按照图 4-11 所示,填写对应于内存映射的 vm_area_struct 描述符中的相应值。
4. 页面和 slab 之间有何种关系?
5. Linux 的 32 位系统不使用页中间目录,这就是说它只有两级页表。所以虚拟地址的前 10 位对应页全局目录 (PGD) 中的偏移量,第 2 个 10 位对应页表 (PTE) 中的偏移量,剩下的 12 位对应页面内的偏移量。

Linux 系统中页面大小是多少? 一个进程可访问多少页,有多大内存可访问?

6. 内存管理区 (memory zone) 与页面之间有何种关系?
7. 从硬件角度看,“实地址”与“虚地址”有何不同?

第5章

输入/输出

本章内容

- 总线、桥、端口和接口的硬件实现
- 设备

Linux 内核可以运行于一个或多个处理器上。处理器为系统中其余部分提供的接口离不开硬件的支持。在硬件依赖性极强的底层，内核使用汇编语言指令与这些设备“对话”。本章分析内核与周围硬件之间的关系，重点放在文件 I/O 和硬件设备上，通过讨论如何将顶层虚拟文件系统的操作实现为底层向物理介质写入二进制位，来分析 Linux 内核如何将软硬件结合起来。

本章首先讨论计算机的核心——处理器，分析它如何与系统中其余部分建立连接，并讨论总线的概念，以及它们如何使处理器与系统中其他部分（例如内存）互连。同时，也介绍绝大多数 x86 系统和 PowerPC 系统都使用的构成芯片组的设备和控制器。

基本理解系统的各个组成部分及其互连后，再来分析系统中软件的层次结构，从上而下依次是：应用程序、操作系统和用于存储的特定块设备——硬盘驱动器及其控制器。虽然下一章才会提到文件系统的概念，但本章也会讨论其部分内容，以便于分析通用块设备层以及块设备最重要的通信方式：请求队列。

本章将在介绍 I/O 调度时，讨论机械设备（硬盘驱动器）和系统软件之间的重要关系。在理解硬盘驱动器的物理几何结构以及操作系统如何划分驱动器的基础上，将进一步分析软件和底层硬件之间的时间选择。

通过进一步剖析硬件，读者还将了解通用块设备驱动程序如何与特定块设备驱动程序通信，从而可以使用通用软件来控制各种硬件设备。最后，在从应用层转到 I/O 层的过程中，还将谈到磁盘控制器所需的硬件 I/O，并给出 I/O 和设备驱动器的其他例子。

接下来，还将讨论另一种主要设备类型——字符设备，以及字符设备与块设备、网络设备有何不同之处。同时，也将一并讨论 DMA 控制器、时钟设备、终端设备等与其他设备相比的重要性。

5.1 总线、桥、端口和接口的硬件实现

处理器与周围设备的通信依赖于一系列电路（或电子线路），总线（bus）就是这样一组具有

类似功能的电子线路。处理器输入/输出最常见的总线类型有：用于对设备寻址（addressing）的总线，在处理器与设备之间传送数据的总线，以及传输控制信息（例如特定于设备的初始化及其特性信息）的总线。因此，可以说设备与处理器通信（反之亦然）主要通过地址总线、数据总线和控制总线来实现。

处理器最基本的功能就是取得并执行指令。这些指令总称为**计算机程序**（computer program）或软件，程序驻留在被称之为内存的一个（或一组）设备中。处理器通过地址总线、数据总线和控制总线来访问内存。执行程序时，处理器通过地址总线来定位指令在内存中的位置，通过数据总线来传输（获取）指令，通过控制总线来确定传输方向（处理器输入/输出）和传输类型（在此，也就是内存）。当提到前端总线（front-side bus）和 PCI 总线之类的特殊总线时，是指地址总线、数据总线和控制总线的总称，这些术语可能容易引起混淆。

软件在系统中运行需要许多外围设备的支持。当前计算机系统两个主要外围设备（也称为控制器）就是**北桥**（Northbridge）和**南桥**（Southbridge）。桥一般是指连接两条总线的硬件设备。图 5-1 说明了北桥和南桥如何与其他设备互连。总之，这些控制器都是系统中的**芯片组**（chipset）。

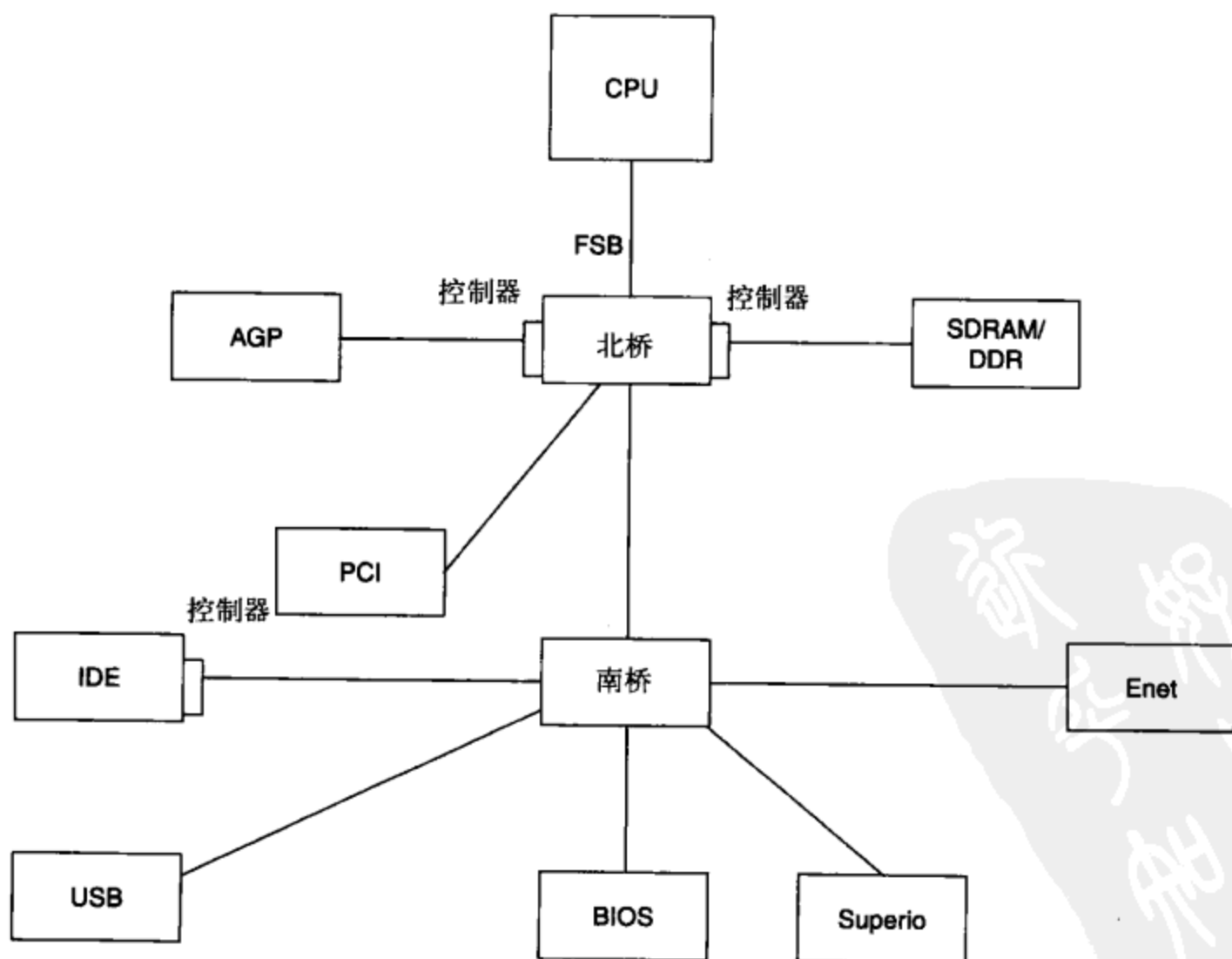


图 5-1 传统的 Intel 系统

北桥连接高速、高性能的外围设备，如内存控制器和 PCI 控制器。然而，某些芯片组在设计

之初就包含集成到北桥的图形控制器。虽然有些芯片组的设计会在北桥中集成图形控制器，但最近的设计大多会包含 AGP (Accelerated Graphics Port, 加速图形接口) 和 PCI Express 之类用来与专用图形适配器通信的高性能总线。为了获得更快的速度和良好的性能, 北桥会在前端总线^①和 (取决于特定芯片组的设计) PCI 总线或内存总线之间架起一座桥梁。

与北桥相连的南桥, 连接着一些低性能的设备。例如, Intel PIIX4 的南桥连接着 PCI-ISA 桥、IDE 控制器、USB、实时时钟、两个 82C59 中断控制器 (详细内容参见第 3 章)、82C54 定时器、两个 82C37 DMA 控制器, 以及 I/O APIC。

最早的 x86 系列个人计算机中, 与键盘、串口、并口等基本外设通讯依赖于 I/O 总线。I/O 总线是一种控制总线, 相对而言, 它与外设通信的速度较慢。x86 体系结构包含特殊的 I/O 指令, 用于在 I/O 总线上传输数据, 如 `inb` (read in a byte, 按字节读入) 和 `outb` (write out a byte, 按字节写出)。I/O 总线是通过共享处理器地址线 and 数据线来实现的。控制线只有在使用特殊的 I/O 指令时才被激活, 以免将 I/O 设备与内存混淆。PowerPC 体系结构控制外设的方式有所不同, 它采用内存映射 I/O (memory-mapped I/O) 方式给设备分配地址空间, 以便控制设备并与之通信。

例如, 在 x86 体系结构中, 第一个并口数据寄存器位于 I/O 端口 0x378 处, 而在 PPC 中, 根据其实现应在存储单元 0xf0000300 处。在 x86 中读取该寄存器的值应执行汇编指令 `in al, 0x378`, 此时为并口控制器激活了一条控制线, 告诉总线 0x378 是 I/O 端口, 而不是内存地址。在 PPC 中读取该寄存器的值则应执行汇编指令 `lbz r3, 0 (0xf0000300)`。并口控制器监视地址总线^②, 并仅当所请求的地址范围在 0xf0000300 以内时才应答。

随着个人计算机日趋成熟, 更多离散的 I/O 设备被整合成单个集成电路, 名曰超级输入输出 (Superio) 芯片。超级输入输出功能块常被进一步合并到南桥芯片 (如 ALIM1543C) 中, SMSC FDC37C932 就是将常见功能整合进离散超级设备的例子, 它包括键盘控制器、实时时钟、电源管理设备、软盘控制器、串口控制器、并口控制器、IDE 接口和通用 I/O 接口。其他南桥芯片包含集成的 LAN 控制器、PCI Express 控制器、音频控制器等。

当前, Intel 的体系结构中已经涉及集线器 (hub) 的概念。现在, 北桥被认为是 GMCH (Graphics and Memory Controller Hub, 图形和内存控制中心), 它支持高性能的 AGP 和 DDR 内存控制器。在 PCI Express 的支持下, Intel 的芯片组正向用于图形的 MCH (Memory Controller Hub, 内存控制中心) 和 DDR2 内存控制器转变。南桥就是所谓的 ICH (I/O Controller Hub, 输入/输出控制器中心)。它们通过一条名为 IHA (Intel Hub Architecture, 英特尔 Hub 架构) 的专有点对点总线互连, 详情参见 Intel 芯片组 865G^③和 925XE^④的数据表。图 5-2 给出了 ICH 的结构。

与英特尔传统的北桥/南桥风格不同, AMD 已经采用主要芯片组元件之间包化的

① 某些 PowerPC 系统中的前端总线也被称为处理器局部总线 (processor-local bus)。

② 也有人把“监视地址总线”叫做“解码地址总线”。

③ <http://www.intel.com/design/chipsets/datashts/25251405.pdf>

④ <http://www.intel.com/design/chipsets/datashts/30146403.pdf>

HyperTransport 技术。对操作系统而言,HyperTransport 与 PCI 兼容^①。详细内容请查阅 AMD 8000 系列芯片组的数据表。图 5-3 解释了 HyperTransport 技术^②。

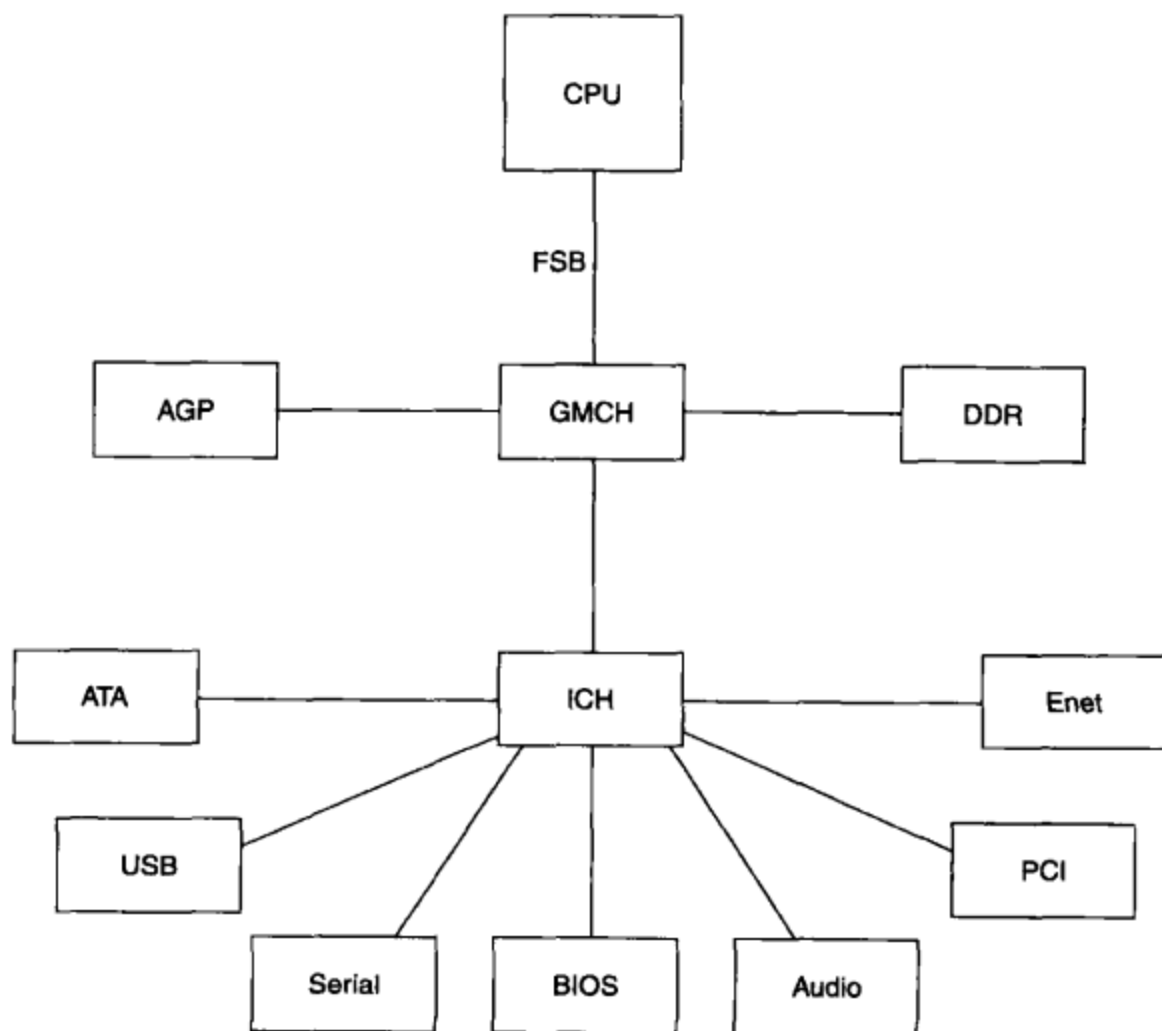


图 5-2 新的英特尔 Hub

采用 PowerPC 体系结构的苹果电脑,其专用设计叫做 UMA (Universal Motherboard Architecture, 通用主板架构)。UMA 希望能在所有 Mac 系统中使用同样的芯片组。

G4 芯片组包括北桥“UniNorth 内存控制器和 PCI 总线桥”,以及南桥“Key Largo I/O 和磁盘设备控制器”。UniNorth 支持 SDRAM、以太网和 AGP。Key Largo 南桥通过 PCI-to-PCI 桥与 UniNorth 互连,它支持 ATA 总线、USB、无线局域网 (WLAN) 和语音。

G5 芯片组包括系统控制器 ASIC (Application Specific Integrated Circuit),它支持 AGP 和 DDR 内存。PCI-X 控制器和高性能 I/O 设备通过 HyperTransport 总线与系统控制器互连。详情请参阅苹果电脑开发者专栏。

简要回顾系统的基本体系结构后,我们再来关注内核为这些设备提供的接口。第 1 章中提到过,设备也被当做文件系统中的文件来处理。和常规文件一样,这类文件也有文件权限、

① 参见 AMD 8000 系列芯片组数据表: http://www.amd.com/us-en/Processors/ProductInformation/0,30_118_6291_4886,00.html。——译者注

② HyperTransport 是一种为主板上的集成电路互连而设计的端到端总线技术,它可以在内存控制器、磁盘控制器以及 PCI 总线控制器之间提供更高的数据传输带宽。HyperTransport 采用类似 DDR 的工作方式,在 400MHz 工作频率下,相当于 800MHz 的传输频率。——译者注

文件模式以及 `open()`、`read()` 等与文件系统相关的系统调用。系统调用的重要性随着要处理的设备的不同而有所不同，也可以为各种类型的设备定制系统调用。因此，设备处理的细节都可以被隐藏于内核中，而对应用级程序员透明。当进程对设备文件应用某一个系统调用时，只需将这个系统调用转换成某种在设备驱动程序中定义的设备处理函数。接下来看看设备的类型。

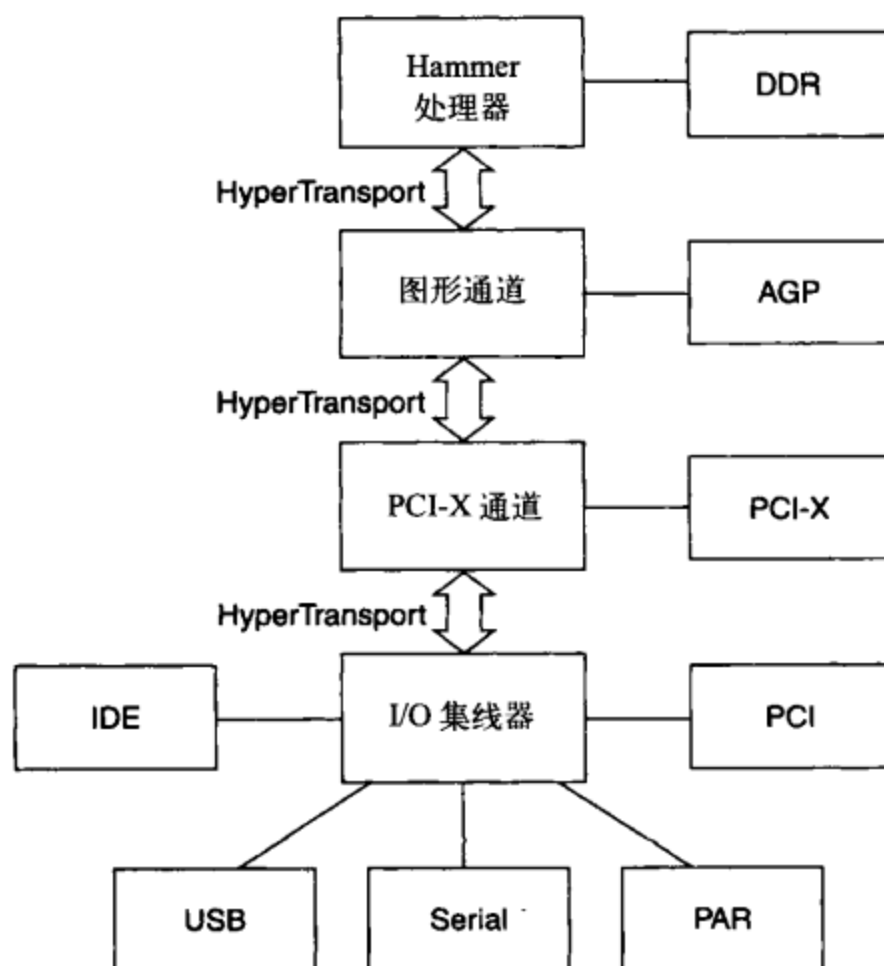


图 5-3 AMD HyperTransport

5.2 设备

系统中有两种设备文件：块设备文件和字符设备文件。块设备每次传送一块数据；顾名思义，字符设备每次只传送一个字符。第三种设备类型是网络设备，它是兼有块设备属性和字符设备属性的特殊设备，但并不用文件来表示。

以前为设备分配设备号时，其主设备号通常用来表示设备驱动程序或设备控制器，从设备号则代表设备控制器所管理的特定设备。现在，这种传统方式已经被名为 `devfs` 的动态方法所取代。这种改变发生的历史原因在于主、从设备号都是 8 位数值，这使得整个系统中仅能有 200 余个静态分配的主设备（块设备和字符设备分别有自己的包含 256 个入口的列表）。/Documentation/devices.txt 中列出了官方已分配的主、从设备号清单。

Linux 内核自 2.3.46 版后就有了设备文件系统（`devfs`）。2.6.7 版内核中并没有默认包含 `devfs`，但可以通过在配置文件中设置 `CONFIG_DEVFS_FS=Y` 将其加入内核。有了 `devfs`，模块

就可以根据名字来注册设备，不需要再使用主/从设备号。考虑到其兼容性，devfs 允许在任何给定的系统中使用主/从设备号，或生成唯一的 16 位设备号。

5.2.1 块设备概述

如前所述，Linux 操作系统将所有设备都看做文件，可以随机引用块设备中任意给定的部分，磁盘驱动器就是一个很好的例子。第一个 IDE 磁盘在文件系统中的名字是 /dev/had，其主设备号是 3，从设备号是 0。通常，磁盘驱动器本身有一个控制器，并且本身具有机电特性（就是说，它有一些可移动部分）。第 6 章将讨论硬盘的基本结构。

通用块设备层

设备驱动程序在驱动程序初始化时注册自身，换句话说，就是将这个驱动程序加入内核的驱动程序表 (driver table) 中，并将设备号映射到数据结构 **block_device_operations** 中。block_device_operations 包含系统中启动和停止一个给定块设备的函数：

```
-----
include/linux/fs.h
760 struct block_device_operations {
761     int (*open) (struct inode *, struct file *);
762     int (*release) (struct inode *, struct file *);
763     int (*ioctl) (struct inode *, struct file *, unsigned, unsigned long);
764     int (*media_changed) (struct gendisk *);
765     int (*revalidate_disk) (struct gendisk *);
766     struct module *owner;
767 };
-----
```

块设备的接口与其他设备类似，函数 open()（见第 761 行）和 release()（见第 762 行）是同步的（换言之，被调用后它们一直运行，直到任务完成）。由于自身机械特性的影响，块设备最重要的函数 read() 和 write() 的实现和常规文件有所不同。我们来分析访问磁盘上的某块数据的情况。从处理器的角度来看，将磁头定位到适当的磁道上并将磁盘转到相应的块要花费相当长的时间，这一等待时间迫使内核实现了系统请求队列 (system request queue)。当文件系统请求一块或多块不在局部页缓存 (page cache) 中的数据时，文件系统将该请求加入请求队列，并将该队列传给通用块设备层，由它来决定机械地检索（或存储）这些信息最有效的方式，然后将它们传给硬盘驱动程序。

最重要的是，初始化时，块设备驱动程序向内核（尤其是块设备管理程序）注册了一个请求队列处理程序，以方便对块设备的读写操作。通用块设备层充当文件系统和设备注册层接口之间的接口，并调优每个读队列和写队列，以便更好地使用新的智能化设备，这是通过带标记的命令队列辅助程序来实现的。例如，如果一个给定队列上的设备支持命令队列，那么，读写操作的优化就可以通过对请求重新排序来实现，使之充分利用底层硬件。此时，队列的调优就体现在能够处理多少个请求的能力上。应用层、文件系统层、通用块设备层和设备驱动程序之间的相互关系参见图 5-4。关于层的更多有用信息以及在早期版本的内核上所作改动的相关信息请参阅文件 biodoc.txt（在目录 /Documentation/block 下）。

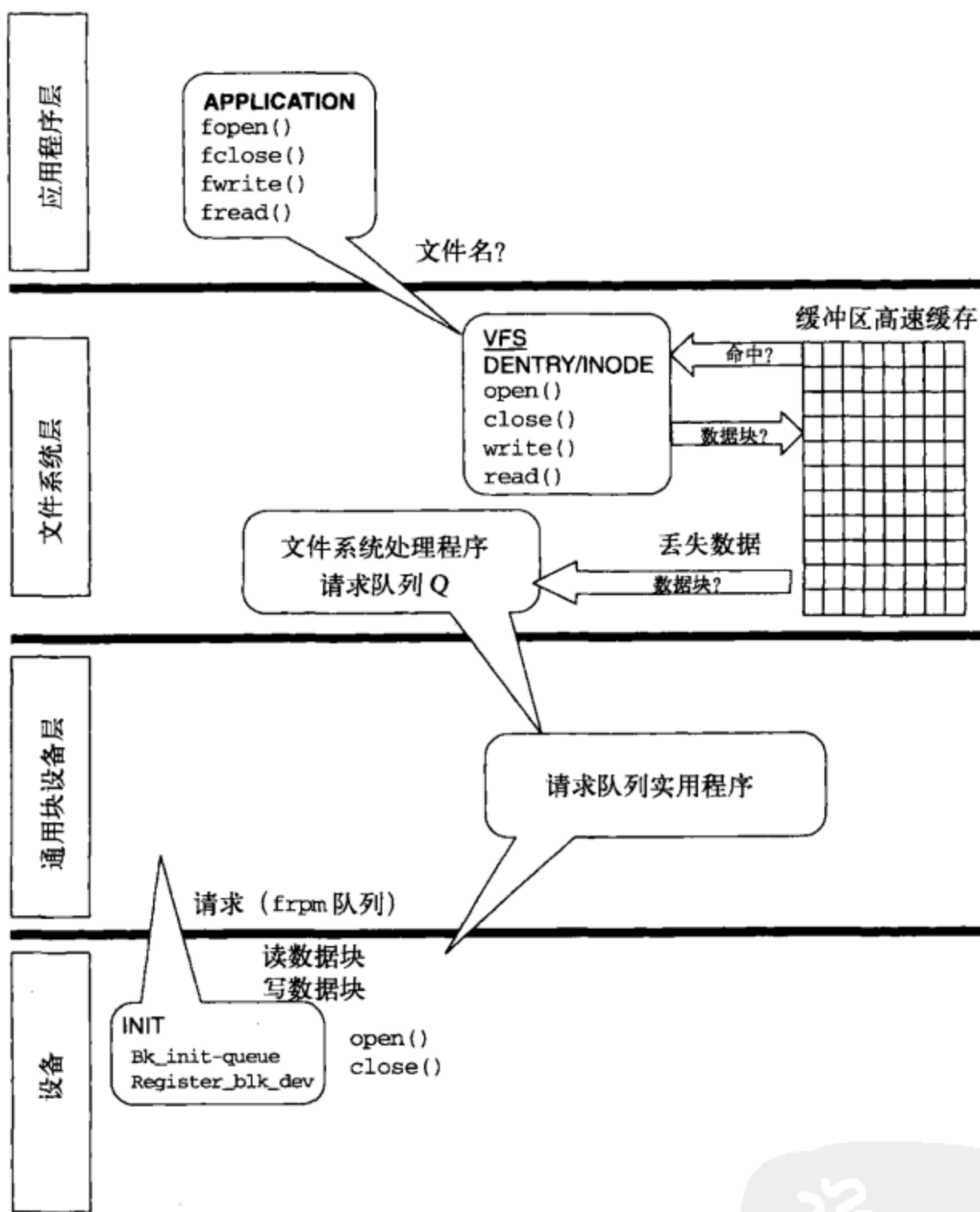


图 5-4 读/写块设备

5.2.2 请求队列和 I/O 调度

读/写请求将从 VFS 出发，通过文件系统驱动程序和页缓存^①，穿越多个层，最后到达块设备驱动程序，并执行实际的设备 I/O 操作，从而获得所请求的数据。

如前所述，块设备驱动程序在初始化时创建并初始化请求队列，同时确定读/写块设备时使用的 I/O 调度算法，即著名的**电梯调度算法** (elevator algorithm)。

内核在启动时默认设置的 I/O 调度算法是**预期调度程序**^② (anticipatory I/O scheduler)。只要将

① 参见第 6 章。

② 某些块设备驱动程序可以在运行时更改其 I/O 调度程序——如果该调度程序在 sysfs 中可见的话。

内核参数 `elevator` 设置成下列值就可以改变 I/O 调度程序的类型。

- (1) **deadline**: 用于最后期限调度程序。
- (2) **noop**: 用于空操作调度程序。
- (3) **as**: 用于预期调度程序。

至此, 能使 I/O 调度程序完全模块化的补丁已经出现了。用户可以使用 `modprobe` 来加载模块, 以及动态地在模块之间进行切换^①, 使用这个补丁, 开始时必须至少有一个调度程序被编译到内核。

在讨论这些 I/O 调度程序如何工作之前, 还必需谈谈请求队列。

块设备使用请求队列来为给定块设备的多个 I/O 请求排序。某些块设备 (如 RAM 盘) 由于其 I/O 请求的开销很小, 可能不需要将请求排队。其他块设备 (如硬盘驱动器) 则因读写开销很大而需要将请求排队。如前所述, 硬盘驱动器的磁头必须在磁道间来回移动, 对 CPU 而言, 每次移动速度之慢简直难以忍受。

为了解决这一问题, 请求队列以优化吞吐量但并不无限推迟请求的方式将块 I/O 读写请求排队。人们常常将 I/O 调度形象地类比为电梯工作的过程^②。如果想让电梯按照提出请求的先后顺序来做出响应的話, 电梯在层与层之间移动的效率就很低, 它也可能直接从最顶层下降到最底层, 中间的各层都不停。如果允许电梯在移动时响应同一方向上的请求, 那么就可以提高运行效率, 同时还能让乘客也满意。同样, 硬盘的 I/O 请求也应该分组组织, 以免磁头来回移动带来过大的开销。前面提到的所有 I/O 调度程序 (空操作 I/O 调度、最后期限 I/O 调度和预期 I/O 调度) 都实现了基本的电梯调度功能, 下面几节将详细讨论。

1. 空操作 I/O 调度程序

空操作 I/O 调度程序^③将接收请求并扫描整个队列, 判断该请求是否能与已有的请求合并。当新请求与某个已有请求要访问的数据块相近时就可以合并。如果请求的是已有请求之前的数据块, 该请求就被合并到这个已有请求之前, 反之被合并到该已有请求之后。在一般的 I/O 操作中, 总是从头开始顺序读取文件的内容, 因此, 大部分请求均可合并到已有请求之后。

如果新请求与已有请求相距较远而不能合并, 空操作 I/O 调度程序就会在队列的已有请求之间寻找适当的位置。如果新请求调用已有请求之间的扇区, 则该请求将被插入到队列中确定的位置。如果找不到适当的位置, 该请求就会被插入到请求队列的队尾。

2. 最后期限调度程序^④

空操作调度程序存在的主要问题是, 有较多相近请求时就永远不会处理新请求。许多与已有请求相近的请求要么被合并, 要么被插入到已有请求之间, 其他新请求就会被堆积在请求队列的队尾。为了解决这个问题, 最后期限调度程序给每个请求分配一个到期时间, 并且用两个附加队列来管理有效时间, 同时用一个与空操作调度算法相似的队列来模拟磁盘效率 (disk efficiency)。

应用程序发出读请求后, 通常会一直等待, 直到该请求被满足才继续执行。但是, 如果发出

① 更多信息, 可在网上搜索 “Jens Axboe” 和 “模块化的 I/O 调度程序”。

② 这一类比正是 I/O 调度程序被称为电梯调度程序的原因。

③ 空操作 I/O 调度程序的代码可参见 `drivers/block/noop-iosched.c`。

④ 最后期限 I/O 调度程序的代码参见 `drivers/block/deadline-iosched.c`。

的是写请求，它并不会一直等到请求被满足为止。应用程序在继续处理其他任务时，写操作就可以在后台执行。因此，最后期限调度程序先响应读请求后响应写请求。除了根据所请求的扇区远近来排序的队列外，调度程序还保持读请求队列和写请求队列，而读、写请求队列中的请求根据时间先后顺序排队（FIFO）。

和空操作 I/O 调度程序一样，当新请求到来时，最后期限调度程序将它放到一个排好序的队列中，根据 I/O 请求的类别，该请求也会被加入到读请求队列或写请求队列。最后期限 I/O 调度程序处理请求时，首先检查读请求队列的队首元素，如果该请求已到期就会被马上处理。类似地，如果无任何读请求到期，调度程序就会去检查写请求队列的队首元素是否到期，如果是，马上处理该请求。仅当无任何读、写请求到期时才去检查标准队列，该队列中请求的处理方式与空操作 I/O 调度算法相似。

读请求的到期时间比写请求的到期时间短，默认时，读请求的到期时间是 0.5 秒，写请求的到期时间是 5 秒。当读请求过多时，到期时间的差异以及对读请求的优先处理可能导致写请求处于饥饿状态，因而需要一个参数通知最后期限 I/O 调度程序，读请求将导致写请求饥饿的最长时间，该参数的默认值为 2。但是，连续请求可以被当做单个请求，因而可以认为，写请求饥饿^①前能处理 32 个连续的读请求。

3. 预期 I/O 调度程序

最后期限调度算法在执行精确的写操作时存在问题。由于重点放在如何使读操作效率最高，所以写请求可以被读请求抢占，使得磁头必须重新定位，读操作完成后返回到写请求，磁头也返回原来的位置。预期 I/O 调度程序^②试图预测出下一个操作，以便提高 I/O 吞吐量。

从结构上来说，预期 I/O 调度程序与最后期限 I/O 调度程序类似，同样有根据时间排序（FIFO）的读请求队列和写请求队列，以及根据扇区远近排序的默认队列。两者的主要区别是，处理完一个读请求后，预期 I/O 调度程序并不马上处理其他请求，它在预测另一个读请求的 6 毫秒内什么都不做。如果该读请求确实发生在相邻区域，它就马上处理该请求。过了预测期后，调度程序又回到正常的操作上来，这时和最后期限调度程序中执行的操作一样。

预测期使得磁头在块设备的扇区之间来回移动带来的 I/O 延迟最小化。

与最后期限 I/O 调度程序类似，预期 I/O 调度算法中也使用了许多参数。读请求的默认到期时间是 1/8 秒，而写请求是 1/4 秒。这两个参数决定了什么时候在诸多读请求和写请求之间进行检查和切换^③。一组读请求 1/4 秒后将检查是否有到期的写请求，一组写请求 1/16 秒后将检查是否有到期的读请求。

默认的 I/O 调度程序是预期 I/O 调度程序，因为对绝大多数应用程序和块设备而言，该调度程序优化了 I/O 吞吐量。如果有数据库方面的应用或需要高磁盘性能的请求，那么使用最后期限 I/O 调度程序会更好一些。空操作 I/O 调度程序常用于 I/O 搜索时间可以忽略不计的系统，例如从 RAM 运行的嵌入式系统。

① 参数定义参见 `deadline-iosched.c` 的第 24~27 行。

② 预测 I/O 调度程序的代码参见 `drivers/block/as-iosched.c`。

③ 参数定义参见 `as-iosched.c` 中的第 30~60 行。

讨论完 Linux 内核的各种 I/O 调度程序后，我们再回过头来讨论请求队列本身以及块设备如何初始化请求队列。

4. 请求队列

在 Linux 2.6 中，每个块设备都有自己的请求队列，以便管理对该设备的 I/O 请求。进程只有获得请求队列锁后才能更新设备的请求队列。以下是 request_queue 结构：

```
-----
include/linux/blkdev.h
270 struct request_queue
271 {
272     /*
273      * Together with queue_head for cacheline sharing
274      */
275     struct list_head queue_head;
276     struct request *last_merge;
277     elevator_t elevator;
278
279     /*
280      * the queue request freelist, one for reads and one for writes
281      */
282     struct request_list rq;
-----
```

第 275 行：指向请求队列队首的指针。

第 276 行：添加到请求队列的最后一个请求。

第 277 行：调度函数（电梯调度程序）通常用于管理请求队列，它可以是某个标准 I/O 调度程序（空操作 I/O 调度程序、最后期限 I/O 调度程序或预期 I/O 调度程序），或为块设备定制的某种新类型的调度程序。

第 282 行：request_list 数据结构由两个 wait_queue 组成，分别用于块设备读请求队列和写请求队列。

```
-----
include/linux/blkdev.h
283
284 request_fn_proc *request_fn;
285 merge_request_fn *back_merge_fn;
286 merge_request_fn *front_merge_fn;
287 merge_requests_fn *merge_requests_fn;
288 make_request_fn *make_request_fn;
289 prep_rq_fn *prep_rq_fn;
290 unplug_fn *unplug_fn;
291 merge_bvec_fn *merge_bvec_fn;
292 activity_fn *activity_fn;
293
-----
```

第 283~293 行：定义与调度程序（或电梯调度程序）相关的函数，以控制如何来管理块设备的请求。

```
-----
include/linux/blkdev.h
```

```

294  /*
295  * Auto-unplugging state
296  */
297  struct timer_list unplug_timer;
298  int unplug_thresh; /* After this many requests */
299  unsigned long unplug_delay; /* After this many jiffies*/
300  struct work_struct unplug_work;
301
302  struct backing_dev_info backing_dev_info;
303
-----

```

第 294~303 行：这些函数常用于“拔掉”块设备的 I/O 调度函数。插入 (plugging) 是指等待更多请求加入请求队列，同时期待更多请求允许调度算法对其排序，以减少执行 I/O 请求所花的时间。例如，硬盘驱动器“插入”一定数目的读请求，期待存在更多读请求时磁头移动较少。这更像是读请求可以顺序排列，或组合成单个大的读请求。拔下 (unplugging) 是指设备决定不再等待并使得系统必须响应其请求的一种方式，而不考虑将来可能的优化。详情可查阅 documentation/block/biodoc.txt。

```

-----
include/linux/blkdev.h
304  /*
305  * The queue owner gets to use this for whatever they like.
306  * ll_rw_blk doesn't touch it.
307  */
308  void *queuedata;
309
310  void *activity_data;
311
-----

```

第 304~311 行：正如内部注释所说的那样，这些行用于请求特定于设备及其设备驱动程序的队列管理。

```

-----
include/linux/blkdev.h
312  /*
313  * queue needs bounce pages for pages above this limit
314  */
315  unsigned long bounce_pfn;
316  int bounce_gfp;
317
-----

```

第 312~317 行：跳跃 (bouncing) 是指内核将高端内存缓冲区的 I/O 请求复制到低端内存缓冲区。在 Linux 2.6 中，内核允许设备在需要时管理高端内存缓冲区。跳跃仅当设备不能处理高端内存缓冲区时才会发生。

```

-----
include/linux/blkdev.h
318  /*
319  * various queue flags, see QUEUE_* below

```

```

320  */
321  unsigned long   queue_flags;
322
-----

```

第318~321行：变量 `queue_flag` 存储一个或多个队列标志，参见表 5-1（参阅 `include/linux/blkdev.h` 的第 368~375 行）。

表5-1 `queue_flags`

标 志 名	功 能
<code>QUEUE_FLAG_CLUSTER</code>	将几个段合并成一段
<code>QUEUE_FLAG_QUEUED</code>	使用通用标记来排队
<code>QUEUE_FLAG_STOPPED</code>	队列被停止
<code>QUEUE_FLAG_READFULL</code>	读队列已满
<code>QUEUE_FLAG_WRITEFULL</code>	写队列已满
<code>QUEUE_FLAG_DEAD</code>	队列被撤销
<code>QUEUE_FLAG_REENTER</code>	避免重入
<code>QUEUE_FLAG_PLUGGED</code>	插入队列

```

-----
include/linux/blkdev.h
323  /*
324  * protects queue structures from reentrancy
325  */
326  spinlock_t   *queue_lock;
327
328  /*
329  * queue kobject
330  */
331  struct kobject kobj;
332
333  /*
334  * queue settings
335  */
336  unsigned long   nr_requests; /* Max # of requests */
337  unsigned int    nr_congestion_on;
338  unsigned int    nr_congestion_off;
339
340  unsigned short   max_sectors;
341  unsigned short   max_phys_segments;
342  unsigned short   max_hw_segments;
343  unsigned short   hardsect_size;
344  unsigned int     max_segment_size;
345
346  unsigned long    seg_boundary_mask;
347  unsigned int     dma_alignment;
348
349  struct blk_queue_tag *queue_tags;
350
351  atomic_t        refcnt;

```

```

352
353 unsigned int    in_flight;
354
355 /*
356  * sg stuff
357  */
358 unsigned int    sg_timeout;
359 unsigned int    sg_reserved_size;
360 };

```

第 323~360 行：这些变量定义了请求队列中可管理的资源，如锁（第 326 行）和内核对象（第 331 行），同时还提供了特殊的请求队列设置，如最大请求数（第 336 行）和块设备的物理约束（第 340~347 行）。如果块设备能使用 SCSI，还可以定义 SCSI 属性（第 355~359 行）。如果想要使用标记的命令队列，则可使用数据结构 `queue_tags`（第 349 行）来实现。`refcnt` 和 `in_flight`（第 351 行和第 353 行）用于计算队列的引用数（通常用于锁）和进程中的动态请求数。

Linux 2.6 内核通过在设备的 `__init` 函数中调用下列函数来初始化块设备的请求队列。在这些函数中，可以看到请求队列内部的细节和相关的辅助例程。相对早期的 Linux 而言，在 Linux 2.6 内核中，每个块设备控制自己的锁，并将自旋锁作为第二个参数传递。第一个参数是块设备驱动程序提供的请求函数。

```

-----
drivers/block/ll_rw_blk.c
1397 request_queue_t *blk_init_queue(request_fn_proc *rfn, spinlock_t *lock)
1398 {
1399     request_queue_t *q;
1400     static int printed;
1401
1402     q = blk_alloc_queue(GFP_KERNEL);
1403     if (!q)
1404         return NULL;
1405
1406     if (blk_init_free_list(q))
1407         goto out_init;
1408
1409     if (!printed) {
1410         printed = 1;
1411         printk("Using %s io scheduler\n", chosen_elevator->elevator_name);
1412     }
1413
1414     if (elevator_init(q, chosen_elevator))
1415         goto out_elv;
1416
1417     q->request_fn    = rfn;
1418     q->back_merge_fn  = ll_back_merge_fn;
1419     q->front_merge_fn = ll_front_merge_fn;
1420     q->merge_requests_fn = ll_merge_requests_fn;
1421     q->prep_rq_fn     = NULL;
1422     q->unplug_fn      = generic_unplug_device;
1423     q->queue_flags    = (1 << QUEUE_FLAG_CLUSTER);
1424     q->queue_lock     = lock;

```

```

1425
1426 blk_queue_segment_boundary(q, 0xffffffff);
1427
1428 blk_queue_make_request(q, __make_request);
1429 blk_queue_max_segment_size(q, MAX_SEGMENT_SIZE);
1430
1431 blk_queue_max_hw_segments(q, MAX_HW_SEGMENTS);
1432 blk_queue_max_phys_segments(q, MAX_PHYS_SEGMENTS);
1433
1434 return q;
1435 out_elv:
1436 blk_cleanup_queue(q);
1437 out_init:
1438 kmem_cache_free(requestq_cachep, q);
1439 return NULL;
1440 }

```

第 1402 行：从内核的内存为队列分配空间，并将其内容初始化为 0。

第 1406 行：初始化包含读队列和写队列的请求清单。

第 1414 行：将所选择的电梯调度程序与该队列关联并初始化。

第 1417~1424 行：将与电梯调度程序相关的函数与该队列关联。

第 1426 行：该函数为段归并设置边界，并检查它是否满足最小尺寸。

第 1428 行：该函数设置驱动程序用于从队列删除请求的函数。它允许使用其他函数来绕过该队列。

第 1429 行：初始化归并段的上限。

第 1431 行：初始化物理设备可处理的最大段数。

第 1432 行：初始化每一请求的最大物理段数目。第 1429~1432 行设置该值，详细内容可参阅 `include/linux/blkdev.h`。

第 1434 行：返回已初始化的队列。

第 1435~1439 行：这是错误事件中用于清除内存的例程。

现在，已经有了位于适当位置并已初始化的请求队列。

在分析通用设备层和通用块设备驱动程序之前，先来简要回顾处理块设备 I/O 操作的软件层（参见图 5-4）。

在应用层，应用程序初始化 `fread()` 等文件操作，虚拟文件系统（VFS）层将接收这一请求（详情请参阅第 4 章）。它保存文件的 `dentry` 结构，并通过 `inode` 结构调用文件的 `read()` 函数。VFS 层试图在缓冲区高速缓存中寻找请求的页面，如果该页面不存在，就调用文件系统处理程序来获取适当的物理块。`inode` 链接到文件系统处理程序，该处理程序又和某一文件系统相关。文件系统处理程序调用请求队列实用程序，是通用块设备层的一部分）为相应物理块和设备创建请求，这一请求被加入请求队列，通用块设备层将维护这个队列。

5.2.3 示例：“通用”块设备驱动程序

我们首先来分析通用块设备层。如图 5-4 所示，通用块设备层位于物理设备层之上、文件系

统层之下。它最重要的工作就是维护请求队列及其相关的辅助例程。

注册设备时使用函数 `register_blkdev(major, dev_name, fops)`。该函数接收被请求的主设备号、块设备名（在 `/dev` 目录下）和指向数据结构 `file operations` 的指针，注册成功后将返回所请求的主设备号。

接下来创建 `gendisk` 数据结构。

头文件 `include/linux/genhd.h` 中定义了函数 `alloc_disk(int minors)`，该函数接收分区号并返回指向数据结构 `gendisk` 的指针。数据结构 `gendisk` 的内容如下：

```
-----
include/linux/genhd.h
081 struct gendisk {
082     int major;      /* major number of driver */
083     int first_minor;
084     int minors;
085     char disk_name[16]; /* name of major driver */
086     struct hd_struct **part; /* [indexed by minor] */
087     struct block_device_operations *fops;
088     struct request_queue *queue;
089     void *private_data;
090     sector_t capacity;
091
092     int flags;
093     char devfs_name[64]; /* devfs crap */
094     int number; /* more of the same */
095     struct device *driverfs_dev;
096     struct kobject kobj;
097
098     struct timer_rand_state *random;
099     int policy;
100
101     unsigned sync_io; /* RAID */
102     unsigned long stamp, stamp_idle;
103     int in_flight;
104 #ifdef CONFIG_SMP
105     struct disk_stats *dkstats;
106 #else
107     struct disk_stats dkstats;
108 #endif
109 };
-----
```

第 82 行：将函数 `register_blkdev()` 的返回值写入 `major_num` 字段。

第 83 行：硬盘驱动器的块设备能够管理几个物理驱动器。虽然块设备依赖于驱动程序，但其从设备号通常用来标识每一个物理驱动器。`first_minor` 是第一个物理驱动器的从设备号。

第 85 行：`hda` 和 `sdb` 之类的磁盘名 (`disk_name`) 是整个磁盘的文本名称（磁盘上的分区分别叫做 `hda1`, `hda2`, 依次类推），是物理磁盘设备上的逻辑磁盘。

第 87 行：`fops` 字段是已初始化为 `file operations` 数据结构的 `block_device_operations`。`file operations` 包含了底层设备驱动程序中指向辅助函数的指针。由于并不是

所有驱动程序中都实现了这些函数，因此这些函数依赖于特定的驱动程序。普遍实现了的文件操作有 open、close、read 和 write。第4章已讨论过 file operations 数据结构。

第 88 行：queue 字段指向驱动程序必须执行的请求操作列表。稍后将讨论请求队列的初始化。

第 89 行：private_data 指针指向那些依赖于特定驱动程序的数据。

第 90 行：capacity 被设置成驱动器的大小（单位是 1 个扇区，512 KB）。调用函数 set_capacity() 可以获得这个数值。

第 92 行：flags 用于标明设备的属性。如果是磁盘驱动器，flags 存储的就是介质类型，例如 CD，可移动介质，等等。

现在，我们来考虑初始化请求队列时要涉及哪些内容。由于该队列已声明，因此，只需调用函数 blk_init_queue(request_fn_proc, spinlock_t) 即可。该函数把代表文件系统的转换函数作为它的第一个参数，并调用 blk_alloc_queue() 来分配队列，同时初始化该队列结构。对所有操作而言，blk_init_queue() 的第二个参数是与队列相关的锁。

最后，要使这个块设备对内核可见，驱动程序还必须调用函数 add_disk()：

```
-----
Drivers/block/genhd.c
193 void add_disk(struct gendisk *disk)
194 {
195     disk->flags |= GENHD_FL_UP;
196     blk_register_region(MKDEV(disk->major, disk->first_minor),
197         disk->minors, NULL, exact_match, exact_lock, disk);
198     register_disk(disk);
199     blk_register_queue(disk);
200 }
-----
```

第 196 行：根据其大小和分区号将设备映射到内核。

调用函数 blk_register_region() 时需要如下 6 个参数。

- 磁盘的主设备号和第一个从设备号。
- 第一个从设备号之后的从设备号范围（如果这个驱动程序管理多个从设备的话）。
- 包含驱动程序的可加载模块（如果有驱动程序的话）。
- 用于查找相应磁盘的例程 exact_match。
- 加锁函数 exact_lock。exact_match 例程找到相应的磁盘以后，该函数就给这些代码加锁。
- 处理程序 disk，用于帮助 exact_match 例程和 exact_lock 函数识别特定磁盘。

第 198 行：register_disk 检查磁盘分区并将其加入文件系统。

第 199 行：为特定区域注册请求队列。

5.2.4 设备操作

基本的通用块设备有 open、close（或 release）和 ioctl 函数，以及最重要的 request 函数。至少，open 和 close 可以单纯作为计数器来使用。ioctl() 接口可以绕过软件层来进行调试和性能测试。文件系统将一个请求加入队列时将调用 request 函数，并抽取出其数据结构，

按照该结构的具体内容进行操作。设备根据该请求是读还是写来执行相应的操作。

对请求队列的访问只能通过一组辅助例程来实现（可参阅 `drivers/block/elevator.c` 和 `include/linux/blkdev.h`）。为了与基本设备模型保持一致，应该在 `request` 函数中加入作用于下一个请求的功能：

```
-----
drivers/block/elevator.c
186 struct request *elv_next_request(request_queue_t *q)
-----
```

这个辅助函数返回指向下一个请求的数据结构的指针，驱动程序可以通过查看其内容来收集所有信息，以便确定其大小、方向以及与该请求相关的任何其他自定义操作。

驱动程序完成该请求后，将通过辅助函数 `end_request()` 向内核报告这一信息：

```
-----
drivers/block/ll_rw_blk.c
2599 void end_request(struct request *req, int uptodate)
2600 {
2601     if (!end_that_request_first(req, uptodate, req->hard_cur_sectors)) {
2602         add_disk_randomness(req->rq_disk);
2603         blkdev_dequeue_request(req);
2604         end_that_request_last(req);
2605     }
2606 }
-----
```

第 2599 行：在请求队列中传递从函数 `elv_next_request()` 获得的参数。

第 2601 行：函数 `end_that_request_first()` 用于传递扇区号（如果扇区尚未确定，则函数 `end_request()` 直接返回）。

第 2602 行：加入系统熵池。熵池是一个系统方法，中断时被调用，从而从函数快速产生随机数，其基本思想是从系统内各种驱动程序中收集数据，并以此来生成随机数。第 10 章将讨论这个问题，文件 `drivers/char/random.c` 的开头部分对此也有解释。

第 2603 行：从队列中删除请求数据结构。

第 2604 行：收集统计信息并释放可用的数据结构。

由此看来，通用驱动程序服务所有请求，直到它被释放为止。

参阅图 5-4，现在，已经有了创建并维护请求队列的通用块设备层。块设备 I/O 系统的最后一层是硬件（或特殊的）设备驱动程序。硬件设备驱动程序在通用设备驱动层使用请求队列辅助例程，来响应已注册的请求队列中的请求，并在完成该请求后发送通知。

硬件设备驱动程序包含与寄存器位置、I/O、定时、中断和 DMA（参见 5.2.9 节）等相关的底层硬件的详细信息。本章并不讨论完整的 IDE 或 SCSI 驱动程序有多复杂，第 10 章将提供更多硬件设备驱动程序的信息，以及一组可帮助你建立驱动程序框架的项目。

5.2.5 字符设备

与块设备不同，字符设备用于传送数据流。所有串行设备都是字符设备。键盘控制器和串行

终端就是典型的字符流设备，使用这些设备时，很明显不能（我们也不想）从中随机访问数据。这就为分组数据传输引入了空白。以太网在物理传输层的介质是串行设备，但在总线级，它将使用 DMA 方式在存储器与外设之间传输数据块。

设备驱动程序设计人员可以让硬件做任何事情，但实时性和实用性才是决定性的因素，因而人们无法随机访问 IDE 驱动器上的音频流和流数据。虽然这两种想法看起来都很有吸引力，但还是必须遵守以下两个简单的规则。

- 所有 Linux 设备 I/O 都基于文件。

- 所有 Linux 设备 I/O 要么是字符设备 I/O，要么是块设备 I/O。

本章最后讨论的并口驱动程序是一个字符设备驱动程序。字符设备驱动程序和块设备驱动程序的相似之处在于：它们都是基于文件 I/O 的接口。表面上，两者都使用 `open`、`close`、`read` 和 `write` 等文件操作。实质上，它们之间最显著的区别就是，字符设备没有读写操作的请求队列的块设备系统（前面已经讨论过）。对没有缓冲区的字符设备而言，每接收到一个元素（字符）就产生一个中断。块设备则在接收一个数据块后才产生一个中断。

5.2.6 网络设备

网络设备兼有块设备和字符设备的一些属性，通常被认为是一组特殊的设备。与字符设备类似，网络设备的数据在物理层上串行传输。但数据包化并通过 DMA（direct memory access，直接存储器存取）方式在网络控制器上传输（参见 5.2.9 节），这一点与块设备相似。

作为 I/O 设备的一种，本章也提到了网络设备，但鉴于其过于复杂，本书不详细讨论。

5.2.7 时钟设备

时钟是用于对系统的硬件检测信号（heartbeat）计数的 I/O 设备。如果没有时间流逝的概念，Linux 就岿然不动了。第 7 章将介绍系统时钟和实时时钟。

5.2.8 终端设备

最早的终端设备是电传打字机（因而串口驱动程序被称为 `tty`）。自从世纪之交人们希望在电报线路上传送实时文本数据以来，电传打字机已经有了极大发展。20 世纪 60 年代初，电传打字机随着早期 RS-232 标准的诞生而日趋成熟，并似乎跟得上不断增长的小型机的发展脚步。为了与计算机进行通信，20 世纪 70 年代，电传打字机不得不让位于计算机终端。真正意义上的计算机终端却是凤毛麟角，从 20 世纪 70 年代到 80 年代中期，随着大型机和微型机的风行，它们又被运行终端仿真软件包的 PC 所取代。终端本身（也称为“哑”终端）只是显示器和通过串行方式与大型机通信的键盘设备。与 PC 不同的是，它的“智能”仅限于发送和接收文本数据。

主控制台（可在引导时配置）是 Linux 系统启动后运行的第一个终端，通常会加载一个图形界面，之后是终端仿真程序窗口。

5.2.9 直接存储器存取

DMA（Direct Memory Access，直接存储器存取）控制器是位于 I/O 设备和（通常是）系统

中高性能总线之间的硬件设备。它的主要作用是在无处理器干预的情况下传输大量数据。DMA 控制器可以看成是专用处理器，可以对它编程以实现和主存之间的大块数据传输。在寄存器级，DMA 控制器只要获得源地址、目的地址和要传送的数据长度就能够完成数据传送任务。然后，当主处理器空闲时，它就可以处理大量数据的传送（从设备到存储器、从存储器到存储器或从存储器到设备）。

许多控制器（例如磁盘控制器，网络控制器和图形控制器）都有内置的 DMA 引擎，因此可以实现大量数据传输而不占用宝贵的 CPU 周期。

5.3 小结

本章描述 Linux 内核如何处理输入和输出操作，更准确地说，讨论了如下几点：

- 简要回顾了 Linux 内核用于执行底层输入输出操作的硬件，如桥和总线；
- 讨论了 Linux 如何表示块设备及如何与块设备交互；
- 介绍了各种 Linux 调度程序和请求队列：空操作 I/O 调度程序、最后期限 I/O 调度程序、预期 I/O 调度程序。

5.4 项目：创建并口驱动程序

本项目简要介绍并口控制器，举例说明前面讨论过的 I/O 例程如何合并。并口通常被集成到芯片组的超级输入输出中，是字符设备驱动程序框架的一个很好的例子。该驱动程序（或可动态加载的模块）并不是非常有用，但可以在此基础上进一步组建并完善其功能。由于是在寄存器级实现对设备的寻址，因此，只要记录了寄存器 I/O 映射，该模块就可以用于访问 PowerPC 系统的 I/O。

本项目的并口设备驱动程序使用标准函数 `open()`、`close()`，最重要的是使用 `ioctl()` 接口来说明该设备驱动程序的结构及其内部工作原理。本项目中并不使用 `read()` 和 `write()` 函数，因为 `ioctl()` 调用已经返回了寄存器的值（因为该设备驱动程序是一个可动态加载的模块，此处仅把它当做一个模块来处理）。

首先，本项目将简要描述如何与并口对话，然后深入研究字符设备驱动程序模块的基本操作。对设备中寄存器的引用通过 `ioctl()` 接口来实现，同时也创建应用程序与该模块进行交互。

5.4.1 并口的硬件

在网上任意搜索“并口”都能得到大量信息。本节的目的是描述 Linux 的模块，因而仅提及这一设备的基本概念。

本项目的实验环境是 x86 系统，驱动程序的框架可以很容易地移植到 PowerPC 中，仅需在 I/O 层与另一设备对话即可。虽然很多嵌入式 PowerPC 系统中都提供了并口，但它并未广泛用于桌面系统（如 G4 和 G5）。

为了真正与并口寄存器通信，要使用 `inb()` 和 `outb()`，它们就像 `readb()` 和 `writb()` 一样易于操作。在 x86 和 PowerPC 中，这两条指令都是在头文件 `io.h` 中定义的。宏 `readb()` 和 `writb()` 都可以处理 x86 和 PowerPC 中的底层 I/O 例程，因而与体系结构无关。

x86 系统中的并口常被当做超级输入输出设备的一部分，或者是添加到系统中的单个 (PCI) 卡。如果进入系统的 BIOS 设置程序，可以看到并口被映射到系统 I/O 地址空间的哪个位置。对 x86 系统而言，并口位于 16 进制地址 0x278、0x378 或 0x3bc 处，并使用中断 IRQ7。这是设备的基地址 (base address)。并口有 3 个 8 位寄存器，其基地址如表 5-2 所示。本例中使用 0x378。

表5-2 并口寄存器

位	7	6	5	4	3	2	1	0	I/O端口地址
数据寄存器 (输出)	D7	D6	D5	D4	D3	D2	D1	D0	0x378 (base+0)
状态寄存器 (输入)	Busy*	ACK	Paper end	Select	Error				0x379 (base+1)
控制寄存器 (输出)					Select*	Init	Auto feed*	Strobe*	0x37A (base+2)

*低有效

数据寄存器存放可写出到连接器针脚的 8 位信息。

状态寄存器存放从连接器输入的信号。

控制寄存器向连接器发送特定的控制信息。

并口连接器是 25 针的 D-shell (DB-25)。表 5-3 列出了这些信号如何映射到连接器的特定针脚。

表5-3 信号与并口连接器针脚之间的关系

信号名	针脚号
Strobe	1
D0	2
D1	3
D2	4
D3	5
D4	6
D5	7
D6	8
D7	9
Acknowledge	10
Busy	11
Paper end	12
Select in	13
Auto feed	14
Error	15
Initialize	16
Select	17
Ground	18~25

警告 并口对静电和过电流都很敏感，不要使用集成的（嵌入主板的）并口，除非：

☐ 你对自己的硬件技能很有信心；

☐ 你确信不会毁坏端口和主板。

强烈建议你在本次实验，甚至所有实验中都使用并口适配器。

执行输入操作时，用 470Ω 电阻将 D7（针 9）跨接到 Acknowledge（针 10），D6（针 8）跨接到 Busy（针 11）。为了监视输出，可以用 470Ω 的限流电阻驱动 LED 和数据针 D0~D4，可以在本地使用老式的打印线缆或 25 针凸模 D-shell 连接器来实现这一功能。

注意 一位优秀的寄存器级程序员应该知道尽可能多的底层硬件知识，包括为特殊并口 I/O 设备找出数据表。在数据表中，你可以找到设备的反向/源电流限制。许多网站都提供了并口的接口方法，包括隔离、扩展信号数和增大/减小电阻。任何想了解 I/O 控制器如何工作的人都必须看看这些内容，但本例并不讨论这些。

本模块通过函数 `outb()` 和 `inb()` 来实现并口寻址。第 2 章曾提到这些函数依赖于平台编译环境，在 x86 系统中可以正确地实现 `in` 和 `out` 指令，在采用内存映射的 I/O 方式的 PowerPC 系统中则是 `lbz` 和 `stb` 指令。可以在头文件 `io.h` 中找到这两个平台的内联代码。

5.4.2 运行在并口上的软件

下面讨论与本项目相关的驱动程序函数。`parll.c`、`Make` 和 `parll.h` 文件完整的程序清单，参见书末。

1. 设置文件操作（fops）

如前所述，该模块使用 `open()`、`close()`、`ioctl()` 以及在前面的项目中讨论过的 `init` 和 `cleanup` 操作。

第一步是设置 `file operations` 数据结构。文件 `/linux/fs.h` 中定义了该数据结构，它列举出所有可能用到的函数，可以选择合适的函数在该模块中来实现。当然，只要实现那些需要用到的操作就可以了，而不是详细列出每一个操作。在网上搜索 C99 和 Linux 模块（Linux module）可以获得这一技术的很多信息。使用该数据结构可将 `open`、`release` 和 `ioctl` 的实现（或入口点）位置告知内核。

```
-----
parll.c
struct file_operations parlport_fops = {
    .open =    parlport_open,
    .ioctl =   parlport_ioctl,
    .release = parlport_close };
-----
```

接下来，就是创建函数 `open()` 和 `close()`。这些函数实际上是虚构的，只是在打开和关闭文件时提供一个标志。

parll.c

```
static int parlport_open(struct inode *ino, struct file *filp)
{
    printk("\n parlport open function");
    return 0;
}

static int parlport_close(struct inode *ino, struct file *filp)
{
    printk("\n parlport close function");
    return 0;
}

-----
```

然后创建 `ioctl()` 函数。注意，以下声明应该放在文件 `parll.c` 的开头：

#define MODULE_NAME "parll"
static int base = 0x378;

parll.c

```
static int parlport_ioctl(struct inode *ino, struct file *filp,
    unsigned int ioctl_cmd, unsigned long parm)
{
    printk("\n parlport ioctl function");
    if(_IOC_TYPE(ioctl_cmd) != IOCTL_TYPE)
    {
        printk("\n%s wrong ioctl type",MODULE_NAME);
        return -1;
    }
    switch(ioctl_cmd)
    {
        case DATA_OUT:
            printk("\n%s ioctl data out=%x",MODULE_NAME,(unsigned int)parm);
            outb(parm & 0xff, base+0);
            return (parm & 0xff);

        case GET_STATUS:
            parm = inb(base+1);
            printk("\n%s ioctl get status=%x",MODULE_NAME,(unsigned int)parm);
            return parm;

        case CTRL_OUT:
            printk("\n%s ioctl ctrl out=%x",MODULE_NAME,(unsigned int)parm);
            outb(parm && 0xff, base+2);
            return 0;

    } //end switch
    return 0;
} //end ioctl

-----
```

函数 `ioctl()` 用于处理用户自定义的命令。本模块向用户提供和并口相关的 3 个寄存器。`DATA_OUT` 命令发送一个数据给数据寄存器 (data register), `GET_STATUS` 命令从状态寄存器 (status register) 读取数据, 最后, `CTRL_OUT` 命令设置端口的控制信号。虽然更好的方式是隐藏 `read()` 和 `write()` 例程后的设备细节, 但该模块主要是为了实验 I/O 操作, 而不是封装数据。

这三个命令都已经在头文件 `parll.h` 中定义好了, 创建时通过使用 `IOCTL` 辅助例程作类型检查来实现。我们使用 `IOCTL` 类型检查宏 `IO (type, number)` 而不是一个整数来表示 `IOCTL` 函数, 这里 `type` 定义为 `p` (并口的类型), `number` 则是 `case` 语句中使用的实际 `IOCTL` 数。在 `parlport_ioctl()` 的开头, 要检查类型是否为 `p`。由于应用程序代码使用相同的头文件作为驱动程序的头文件, 因而接口能保持一致。

2. 设置模块初始化例程

初始化模块将该模块与操作系统联系起来, 需要时也可以用于提前初始化任何数据结构。由于这个并口驱动程序不需要用到复杂的数据结构, 因此只注册了该模块。

```
-----
parll.c
static int parl_init(void)
{
    int retval;

    retval= register_chrdev(Major, MODULE_NAME, &parlport_fops);
    if(retval < 0)
    {
        printk("\n%s: can't register",MODULE_NAME);
        return retval;
    }
    else
    {
        Major=retval;
        printk("\n%s:registered, Major=%d",MODULE_NAME,Major);

        if(request_region(base,3,MODULE_NAME))
            printk("\n%s:I/O region busy.",MODULE_NAME);

    }
    return 0;
}
-----
```

函数 `init_module()` 负责向内核注册该模块。函数 `register_chrdev()` 接收请求的主设备号 (5.2 节已经讨论过, 稍候第 10 章还会再次讨论。若主设备号为 0, 则内核将为该模块动态分配一个号)。回想一下, 主设备号是保存在 `inode` 数据结构中, 数据结构 `dentry` 指向 `inode`, 而指向 `dentry` 的指针则存储在一个文件结构中。第二个参数是出现在 `/proc/device` 中的设备名。第三个参数就是上面提到的 `file operations` 数据结构。

成功注册模块之前, `init` 例程用并口基地址和将要操作的寄存器范围的长度 (单位为字节) 来调用函数 `request_region()`。

如果注册失败, 函数 `init_module()` 就会返回一个负数。

3. 设置模块的清理例程

函数 `cleanup_module()` 负责注销模块，并释放先前请求的 I/O 范围：

```
-----
par11.c

static void par11_cleanup( void )
{
    printk("\n%s:cleanup ",MODULE_NAME);
    release_region(base,3);
    unregister_chrdev(Major,MODULE_NAME);
}
-----
```

最后，将所需的 `init` 和 `cleanup` 入口点包括进来：

```
-----
par11.c
module_init(par11_init);
module_exit(par11_cleanup);
-----
```

4. 插入模块

现在可以像前一个项目那样将模块插入到内核中了，所用命令如下：

```
Lkp: ~# insmod par11.ko
```

查看 `/var/log/messages`，发现 `init()` 例程的输出就像前面所说的那样，但返回了更详细的主设备号信息。

前一个项目仅向内核插入模块并从内核移除该模块。现在，需要使用命令 `mknod` 将模块与文件系统联系起来。在命令行输入如下命令：

```
Lkp: ~# mknod /dev/par11 c <xxx> 0
```

其中参数说明如下。

- **c**: 创建特殊的字符设备文件（与块设备文件相对）。
- **/dev/par11**: 到该设备的路径（供 `open` 调用使用）。
- **xxx**: 执行初始化时返回的主设备号（从 `/var/log/messages` 返回）。
- **0**: 该设备的从设备号（本例中没有用到）。

例如，若在 `/var/log/messages` 中看到的主设备号是 254，则该命令看起来可能是这样：

```
Lkp:~# mknod /dev/par11 c 254 0
```

5. 应用程序代码

此处，创建一个简单的应用程序来打开模块，并在输出引脚 D0~D7 上进行二进制计数。使用命令 `gcc app.c` 编译该代码，可执行输出默认为 `a.out`：

```
-----
app.c
000 //application to use parallel port driver
```

```
#include <fcntl.h>
#include <linux/ioctl.h>
004  #include "parll.h"

main()
{
    int fptr;
    int i,retval,parm =0;

    printf("\nopening driver now");
012  if((fptr = open("/dev/parll",O_WRONLY))<0)
    {
        printf("\nopen failed, returned=%d",fptr);
        exit(1);
    }

018  for(i=0;i<0xff;i++)
    {
020      system("sleep .2");
021      retval=ioctl(fptr,DATA_OUT,parm);
022      retval=ioctl(fptr,GET_STATUS,parm);

024      if(!(retval & 0x80))
          printf("\nBusy signal count=%x",parm);
          if(retval & 0x40)
027              printf("\nAck signal count=%x",parm);
028      //  if(retval & 0x20)
//      printf("\nPaper end signal count=%x",parm);
//      if(retval & 0x10)
//      printf("\nSelect signal count=%x",parm);
//      if(retval & 0x08)
033  //      printf("\nError signal count=%x",parm);

        parm++;
    }
038  close(fptr);
}
```

第 4 行：应用程序和驱动程序的头文件通常都包含新的 IOCTL 辅助宏来进行类型检查。

第 12 行：打开驱动程序，获得模块的文件句柄。

第 18 行：进入循环。

第 20 行：减缓循环，以便监视 lights/count。

第 21 行：使用文件指针发送 DATA_OUT 命令到该模块，该模块又使用 outb() 将参数的低 8 位数据写入数据端口。

第 22 行：使用 ioctl 函数的 GET_STATUS 命令读取状态字节。这一过程通过 inb() 来实现，并返回读到的字节数。

第 24~27 行：监视感兴趣的特殊数据位。要注意的是，Busy* 是一个低有效的信号，因而当 I/O 完成时读到的这个值为真。

第 28~33 行：改进设计时取消这些注释。

第 38 行：关闭模块。

如果你已经建立了如图 5-5 所示的连接器，当计数器的最高两位数据置位时，产生了 busy 和 ack 信号。应用程序读取这些数据位并进行相应地输出。

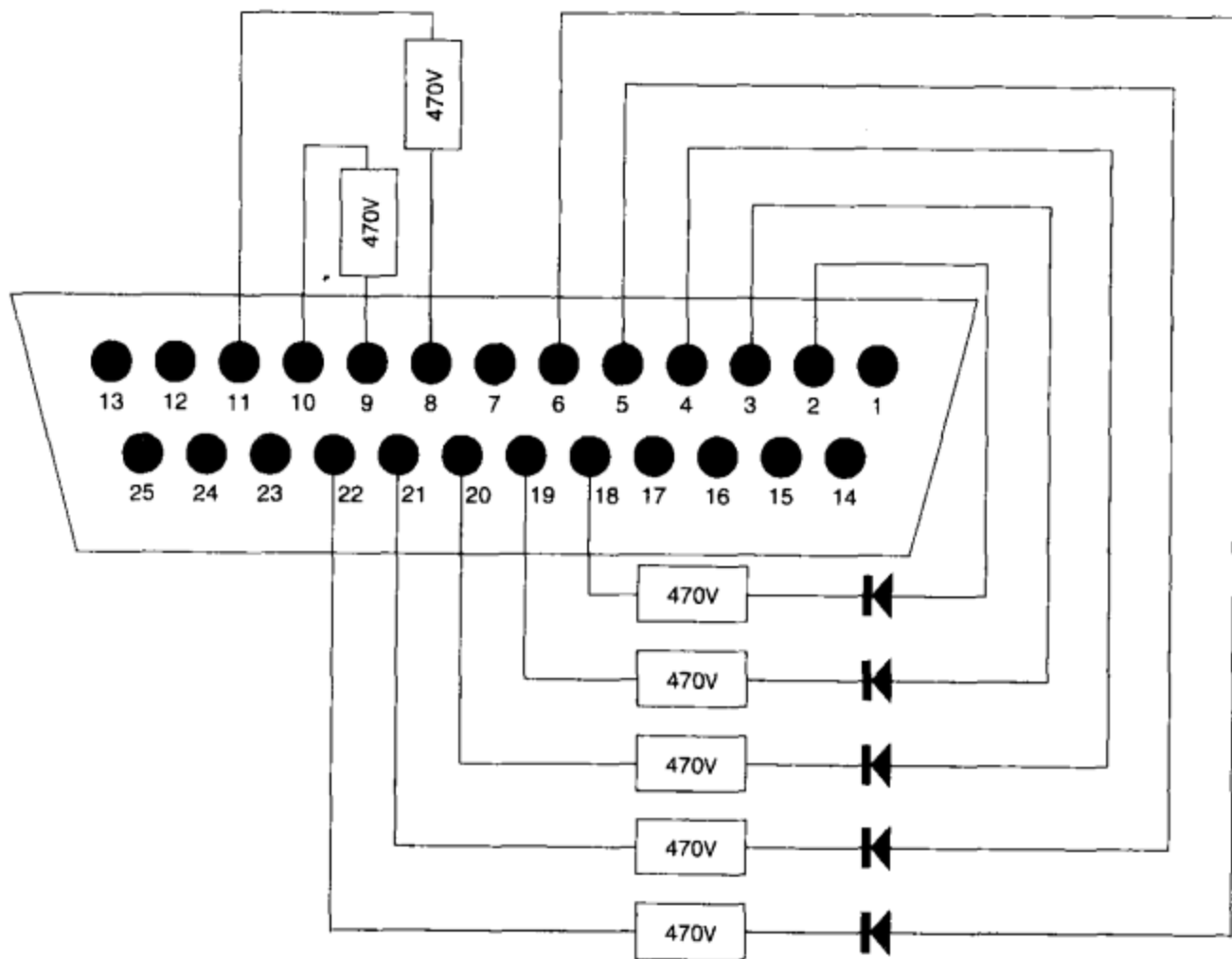


图 5-5 连接器

以上仅简略叙述了字符设备驱动程序的主要内容。了解这些函数后，跟踪工作代码或创建自己的驱动程序都变得简单起来。当给该模块添加中断处理程序时，涉及对函数 `request_irq()` 的调用，同时还要传递所需的 IRQ 号和中断处理程序的名字。这些工作都应该在函数 `init_module()` 中来完成。

建议在该驱动程序中添加如下内容。

□ 使并口模块服务定时器中断轮询输入。

- 怎样将多路 8 位 I/O 数据扩展到 16 位，32 位，64 位？要付出什么代价？
- 从模块的写例程发送字符到串口。
- 使用 ack 信号增加一个中断例程。

5.5 习题

1. 加载一个模块。该模块在文件系统中会变成什么类型的设备文件？
2. 找出已加载的设备文件的主设备号和从设备号。

3. 什么时候采用最后期限 I/O 调度程序比预期 I/O 调度程序更有利一些?
4. 什么时候采用空操作 I/O 调度程序比预期 I/O 调度程序更好一些?
5. 北桥控制器和南桥控制器各有什么特点?
6. 将诸多功能块加入超级输入输出芯片中有何好处?
7. 为什么现在没有将图形和网络通讯功能块加入到超级输入输出芯片中?
8. ext3 之类的日志型文件系统与 ext2 之类的标准文件系统相比, 其主要差别是什么? 有何优势?
9. 预期 I/O 调度程序的基本理论是什么? 这种方法更适合硬盘驱动器和 RAM 盘吗?
10. 块设备和字符设备的主要区别是什么? 各举一例。
11. DMA 是什么? 为什么说它是一种数据传输的有效方式?
12. 电传打字机最初用在何处?



第 6 章

文件系统

本章内容

- 文件系统的基本概念
- Linux 虚拟文件系统
- 与 VFS 相关的结构
- 页缓存
- VFS 的系统调用和文件系统层

信息处理是围绕信息的存储、检索以及操作而展开的。

在第 3 章中，我们对进程是程序执行的基本单元这一主题进行了讨论，对进程处理信息的方式是把自己的实体存放在其地址空间这一形式进行了考察。然而，进程的地址空间只存在于进程的整个生命周期中，而且它只占系统内存大小的一小部分。文件系统的发展源于对大容量、非易失性信息存放在外存而不是存放在寄存器和内存这样的需求。非易失性信息指始终存在的数据，不管处理信息的进程是否结束，不管操作系统是否停止。

信息存放在外存又引出信息如何表示的问题。信息存储的基本单元是**文件**（file）。**文件系统**（filesystem）或文件管理子系统，是操作系统的组成部分，它处理文件结构、文件操作和文件保护。本章就有关 Linux 文件系统实现的问题展开讨论。

6.1 文件系统的基本概念

首先，我们描述 Linux 文件系统所涵盖的概念。大多数人对这些概念已经相当熟悉，因为在 Linux 的使用及用户空间应用程序编程中都涉及这些概念。如果你已经掌握了文件系统的基本概念，可以直接跳到 6.2 节。

6.1.1 文件和文件名

“文件”这个单词是从现实世界借用的术语。自电子管出现以来，信息就存储在文件中。现实世界的文件是由一张或几张预定大小的纸组成的。这些文件通常存放在橱柜中。

在 Linux 中，文件是一种线性的字节流。这些字节的含义对于操作系统来说并不重要，但是对于用户来说至关重要，就像橱柜对文件的内容毫不关心一样。文件系统提供存储数据的用户接口，透明地操作来自外部设备的物理数据。

Linux 中的文件有许多属性和特征。用户最为熟悉的属性通常是文件的名称。文件的名称常常能表明文件的内容。文件名可能含有**文件扩展名**，扩展名是用句号添加到主文件名之后的附加名。这种扩展名为辨别用户空间应用程序的内容提供了额外的方法。例如，迄今为止，我们看到的所有例子的文件扩展名都是.h 或.c。用户空间的程序，例如编译器和链接器，分别用.h 和.c 表示文件是头文件或源文件。

尽管文件名对用户应用（例如编译器）来说可能很重要，但是操作系统毫不关心文件名，因为它只把文件视为容纳字节的容器，而不考虑文件的内容或用途。

6.1.2 文件类型

Linux 支持多种文件类型，包括普通文件、目录文件、链接文件、设备文件、套接字及管道文件。**普通文件**（regular file）包括二进制文件和 ASCII 文件。ASCII 文件仅仅是一些文本行，用户无需解释程序就能看到和读懂它。某些 ASCII 文件是可执行的，称为**脚本**（script）。这些文件由解释程序来执行。从根本上来讲，shell 也是一个解释程序。可执行的二进制文件是非 ASCII 文件，其内容表面上看起来像是随机数据。这些文件有一个内部格式，内核通过解释这个格式来运行程序。这个格式称为**目标文件格式**（object file format），每个操作系统都解释预定义的目标文件格式。第 9 章将更详细地介绍目标文件格式。

在 Linux 中，文件被组织成多级目录系统，比如图 6-1 所显示的系统。**目录**（directory）包含文件，用来维护文件系统的结构。以下内容会更详细地考察目录和 Linux 的文件结构。

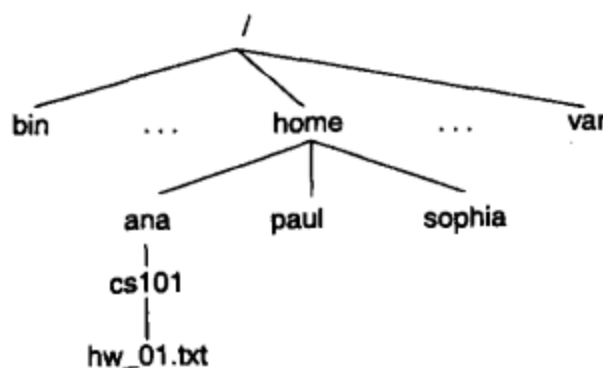


图 6-1 文件系统的层次

链接（link）是指向另一个文件的文件，即文件指针。这些文件仅含有访问另一个文件的必要信息。

设备文件（device file）是 I/O 设备的表示，通过设备文件可以对硬件设备进行访问。程序需要访问 I/O 设备时，使用文件中对应的属性就可以操作文件所代表的设备。有两种主要类型的设备：**块设备**和**字符设备**。块设备（block device）以块为单位传送数据；字符设备（character device）以字符为单位传送数据。第 5 章已介绍了 I/O 设备的详细内容。

套接字文件和**管道文件**是进程间通信的方式。这些文件支持进程间的定向数据流动。我们不讨论这些特殊文件。

6.1.3 文件的附加属性

除了文件名、文件类型和文件数据外，文件还有更多的属性。操作系统会为每个文件添加附

加信息，例如文件的访问权限。在多用户系统（例如 Linux）中，文件保护变得越来越重要。用户被分成三类：

- 用户或文件的所有者；
- 组用户或与文件所有者同组的用户；
- 其他用户，指系统中所有不属于文件组的其余用户。

上述每一类用户对文件分别拥有一组特定的权限。尽管对文件可以进行许多操作，但 Linux 将这些操作归纳为三类并分别对应于三种权限：读、写和执行。因为这三种权限均可应用于三类用户中的每一类，因此，一个文件可能会拥有 9 种不同的权限组合。

文件的其他属性包括文件大小、创建时间戳和最后一次访问的时间戳，所有这些属性都可以通过核心实用程序 `ls` 显示出来。当我们研究文件在内核如何实现时，我们会看到用户看不见的其他许多属性。

6.1.4 目录和路径名

目录是用来维护文件系统层次结构的文件。目录中存放它所含有文件、下级目录以及目录本身的相关信息。在 Linux 中，每个用户都有自己的“主目录”，用户把自己的文件存放在这个目录下，并在这个目录下创建他自己的目录树结构。在图 6-1 中，我们可以看到，目录构成了文件系统的树形结构。

把文件系统组织成树形结构后，只凭文件名就很难找到文件了。我们必须知道文件在树中所处的位置才能找到它。文件的**路径名**（pathname）描述了文件的位置。文件的位置可以相对于树的根目录来描述，这种路径名称为**绝对路径名**（absolute pathname）。绝对路径名从树的根目录（也就是 `/`）开始，目录节点使用目录名后跟 `/` 来表示，例如 `bin/`。因此，文件的绝对路径名就是由一系列目录节点组成的，沿着这些节点在文件系统的树形结构中一层一层下探，最终就会找到相应的文件。在图 6-1 中，文件 `hw1.txt` 的绝对路径名是 `/home/ana/cs101/hw1.txt`。表示文件的另一个方法是用**相对路径名**（relative pathname）。相对路径名依赖于和文件相关的进程的工作目录。工作目录（working directory），或当前目录（current directory），指与进程执行相关的目录。因此，如果 `/home/ana/` 是进程的工作目录，我们就可以把文件称为 `cs101/hw1.txt`。

在 Linux 中，目录包含那些在操作系统运行期间执行不同任务的文件。例如，共享文件存放在 `/usr` 和 `/opt` 目录下，而非共享的文件存放在 `/etc/` 和 `/boot` 目录下。同样地，非静态文件（文件内容可被系统程序修改的文件）存放在 `/var` 下面的 `vcertain` 目录下。关于文件系统层次标准的更多信息请参见 <http://www.pathname.com/fhs>。

在 Linux 中，每个目录有两个与其相关的入口：`.`（发音为“点”）和`..`（发音为“点点”）。入口`.`表示当前目录，而`..`表示其父目录。对根目录来说，`.`和`..`都表示当前目录。（换句话说，根目录是它自己的父目录。）这种表示法使得用以下方式表示相对路径更为方便了。在前面的例子中，工作目录是 `/home/ana`，我们文件的相对路径名是 `csw101/hw1.txt`。如果要在我们的工作目录中表示 `paul` 目录中的文件 `hw1.txt`，就可以使用 `../paul/cs101/hw1.txt` 这个相对路径名，开头的`..`表示先向上返回一层。

6.1.5 文件操作

文件操作包括系统所允许对文件执行的所有操作。一般来说,对文件的操作包括创建和撤消、打开和关闭、读和写,此外,还能对文件重命名、更改它的属性。文件系统提供系统调用,作为完成这些操作的接口,这些系统调用又被置入封装函数中,用户空间的程序可经由链接库来访问这些封装函数。当我们考察 Linux 文件系统的实现时,会深入探究其中的部分操作。

6.1.6 文件描述符

文件描述符就是一个整型数,系统以此来识别打开的文件。`open()` 系统调用返回一个文件描述符,当进程以后要访问这个文件时,对文件的所有操作都可以使用这个文件描述符。在下一节中,我们会看到在内核术语中文件描述符所代表的含义。

每个进程包含一个文件描述符数组。当我们讨论内核为文件系统提供的数据结构时,我们会看到如何用数组管理文件描述符。依照惯例,数组的第一个元素(文件描述符 0)对应进程的标准输入,第二个元素(文件描述符 1)对应标准输出,而第三个元素(文件描述符 2)对应标准错误输出。这就允许应用程序打开标准输入、标准输出及标准错误文件。图 6-2 说明了属于进程的文件描述符数组。

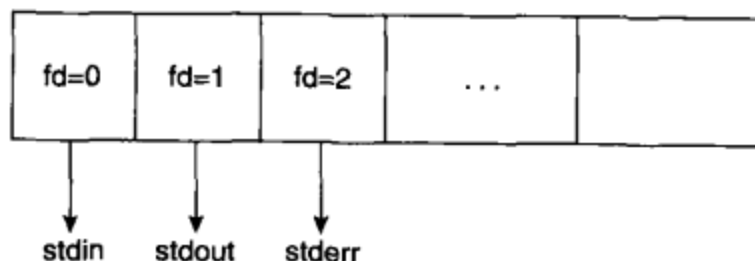


图 6-2 文件描述符数组

文件描述符的分配是以“最小可用索引”的基本原则进行的。因此,如果进程打开多个文件,为它分配的文件描述符就是递增的,如果以前打开的文件在新文件打开之前关闭,就不一定递增。我们可以看到 `open` 系统调用和 `close` 系统调用为了确保这个原则将会如何操作文件描述符。因此,在进程的整个生命期中,进程可能拥有两个不同的文件,但对应相同的文件描述符,这种情况便是先关闭一个文件,然后又开另一个文件。相反,两个不同的文件描述也可能指向同一个文件。

6.1.7 磁盘块、磁盘分区以及实现

为了关注文件系统的实现,我们必须理解关于硬盘的一些基本概念。硬盘利用磁性记录数据。硬盘含有记录数据的多个旋转式盘片。磁头被安装在可在盘片表面来回移动的机械臂上,通过沿着磁盘半径的移动来读写数据,就好像唱片机的针一样。而磁盘片本身像唱片机上面的唱片一样旋转。每张盘片分成多个叫做磁道的同心环。磁道由外向里进行编号。磁道号相同的一组磁道(跨越所有磁盘)叫做柱面。每个磁道(通常)又被分成 512 字节的扇区。柱面、磁道、磁头构成了硬盘驱动器的几何特性。

空磁盘必须在文件系统建立前进行格式化,格式化会创建磁盘的磁道、盘块和分区。分区指

逻辑磁盘，操作系统利用它分配或使用硬盘。我们可以借助于分区把单个磁盘划分为多个部分，使之看起来好像有多个磁盘一样。这使得不同的文件系统可以存在于同一张盘上。每个分区又被分成磁道和块。磁盘中**磁道和块**的创建是由 fdformat^①这样的程序完成的，而逻辑分区的创建由 fdisk 这样的程序完成。磁道和块的创建都是在实际的文件系统创建之前就完成的。

Linux 的文件树可以提供对多个文件系统的访问。这意味着如果你有一个多分区的磁盘，而每个分区都有一个文件系统的话，那么，从一个逻辑命名空间就可能访问到所有这些文件系统。使用 mount 命令可以把每个文件系统加入到 Linux 的主文件系统树中。我们说一个文件系统被安装是指设备文件系统被加入主树，从而可以从主树中访问设备文件系统。文件系统被安装到目录上^②。安装文件系统的目录叫做挂载点 (mount point)。

文件系统实现的主要难点之一在于抉择操作系统对组成文件的字节序列到底如何存放。如前所述，磁盘的分区空间被划分为叫做块的一簇簇空间。块的大小因实现的不同而不同。块的管理方式决定文件的访问速度和碎片的等级^③，并因此而浪费一些空间。例如，如果我们有一个大小为 1 024 字节的块和一个大小为 1 567 字节的文件，显然，这个文件横跨两个块。操作系统通过维护一个叫做索引节点 (inode) 结构中的信息，就可以记录某个文件中块的情况。

6.1.8 性能

文件系统具有提高系统性能的各种方法，其中之一就是通过在内核中维护内部基础结构来快速访问一个给定路径名对应的索引节点。当我们解释文件系统的实现时，将会看到内核是如何实现这种方法的。

页缓存是文件系统提高性能的另一个方法。页缓存是内存中页的集合。它被设计成缓存许多不同类型的页，这些页来源于磁盘文件、内存映射文件或内核能够访问的其他任何页对象。这种缓存机制大大减少了访问磁盘的次数，提高了系统性能。本章将阐述在文件管理的过程中，页缓存是如何与磁盘访问相结合的。

6.2 Linux 虚拟文件系统

文件系统的实现因系统的不同而不同。例如，在 Windows 和 Unix 中，文件与磁盘块如何关联的具体实现就有很大的不同。实际上，微软实现了各种操作系统对应的多种文件系统：DOS 和 Win 3.x 对应的 MS-DOS，Windows 9x 对应的 VFAT，Windows NT 对应的 NTFS。UNIX 操作系统也实现了多种文件系统，例如 SYSV 和 MINIX。特别是，Linux 使用 ext2、ext3 和 ReiserFS 之类的文件系统。

Linux 最具特色的一点就是支持多种文件系统。你不仅能够从它自己的文件系统中 (ext2、ext3、和 ReiserFS) 查看文件，而且能从与其他操作系统相关的文件系统中查看文件。在单独的 Linux 系统中，你能够访问许多不同格式的文件。表 6-1 列出了 Linux 目前支持的文件系统。对

① fdformat 用于软盘的低级格式化 (建立磁道和扇区)。IDE 硬盘和 SCSI 硬盘通常在出厂时就进行了预格式化。

② 用树的说法，你就可以说把子树添加到主树的一个节点上。

③ 第 4 章中介绍了碎片，以及内存中的空洞是如何产生的，相同的碎片问题在硬盘存储中也会遇到。

用户来说,从一种文件系统跨越到另一种文件系统没有任何差异;他能随心所欲地把支持的文件系统安装到最初的那个树命名空间。

表6-1 Linux支持的部分文件系统

文件系统名	说 明
ext2	第二扩展文件系统
ext3	ext3日志文件系统
Reiserfs	日志文件系统
JFS	IBM的日志文件系统
XFS	SGI Irix的高性能日志文件系统
MINIX	Linux最初的文件系统, minix操作系统的文件系统
ISO9660	CD-ROM文件系统
JOLIET	微软的CRDOM文件系统扩展
UDF	可选的CROM、DVD文件系统
MSDOS	微软的磁盘操作系统
VFAT	Windows 95的虚拟文件分配表
NTFS	Windows NT、2000、XP、2003的文件系统
ADFS	Acorn磁盘文件系统
HFS	Apple的Macintosh文件系统
BEFS	BeOs文件系统
FreeVxfs	Veritas Vxfs支持
HPFS	OS/2支持
SysVfs	系统V的文件系统支持
NFS	网络文件系统支持
AFS	Andrew文件系统 (也是网络文件系统)
UFS	BSD的文件系统支持
NCP	NetWare文件系统
SMB	Samba

Linux 不但支持多种磁盘文件系统,还支持网络安装的文件系统,以及用于处理某些专门的事情而不是管理磁盘空间的特殊系统。例如, `procfs` 就是一个伪文件系统。这个虚拟的文件系统提供关于系统各个方面的信息。`procfs` 文件系统不占据硬盘空间,它的文件在访问时动态创建。另一个这样的文件系统是 `devfs`^①,它为设备驱动程序提供接口。

通过在用户空间和实际文件系统之间引入一个抽象的中间层, Linux 完成对实际文件系统细节的“伪装”。这个层称为 VFS (virtual filesystem, 虚拟文件系统)。它把文件系统所特有的结构和来自内核其余部分的功能隔离开来。VFS 管理与文件系统相关的系统调用,并把它们转化成适当文件系统类型的函数。图 6-3 概括了文件系统管理的结构。

① 在 Linux 2.6 中,尽管仍然少量地支持 `devfs`,但是它被 `udev` 取代了。有关 `udev` 的更多信息,参见 <http://www.kernel.org/pub/linux/utils/kernel/hotplug/udev-FAQ>。

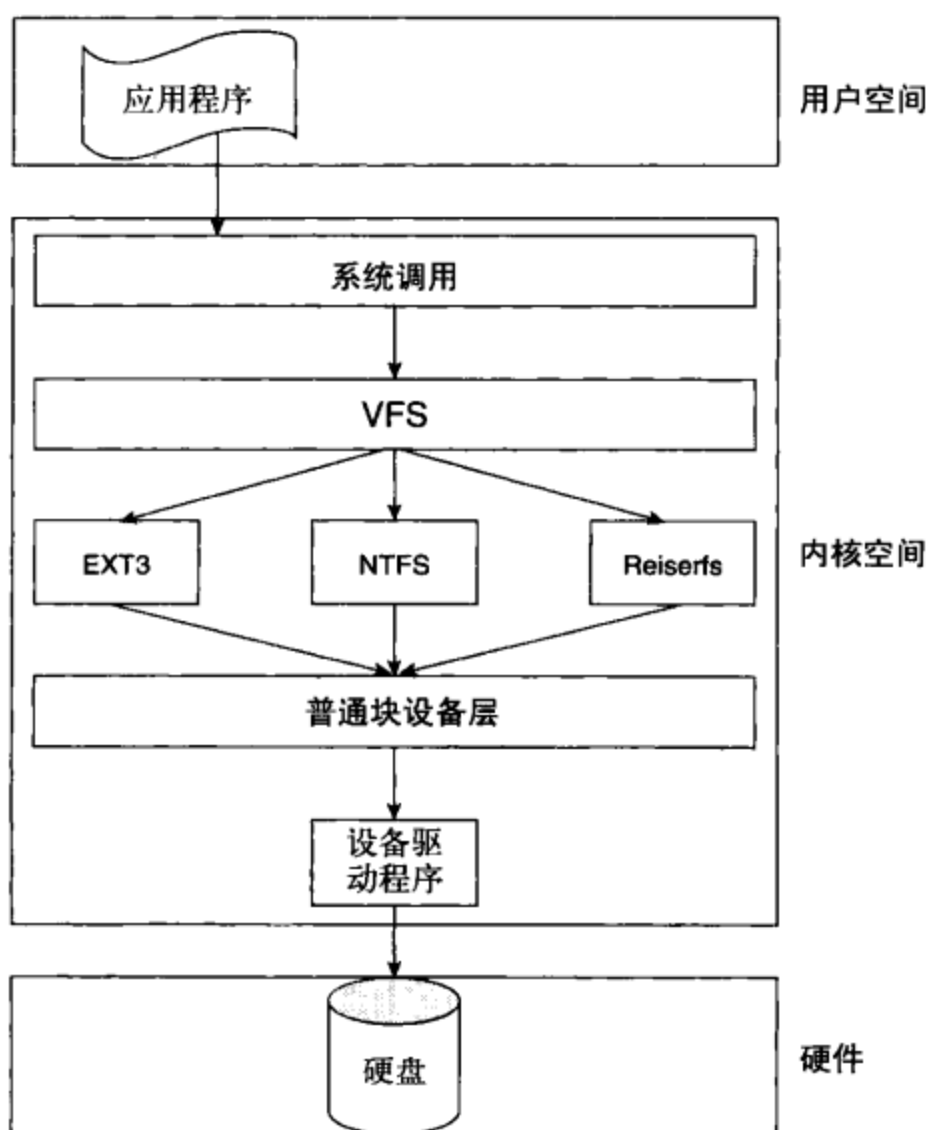


图 6-3 Linux 的虚拟文件系统

用户应用程序通过系统调用访问通用的 VFS。每个被支持的文件系统必须实现一组函数，这组函数完成 VFS 所支持的操作（例如，打开、读、写和关闭文件）。VFS 记录它所支持的文件系统，并给出执行每个操作的函数。从第 5 章可以知道，在文件系统和实际的设备驱动程序之间有一个普通的块设备层。这提供了一个抽象层，使得文件系统特有代码的实现与它最终访问的具体设备无关。

6.2.1 VFS 的数据结构

VFS 利用数据结构来表示文件系统。

该数据结构如下所示。

- 超级块结构。存放已安装的文件系统的相关信息。
- 索引节点结构。存放有关文件的信息。
- 文件结构。存放被进程打开的文件的相关信息。
- 目录项结构。存放有关路径名及路径名所指向的文件的信息。

除了这些结构外，VFS 还使用一些附加结构，例如 `vfsmount` 和 `nameidata`，它们分别存放安装信息和路径的查找信息。尽管我们不会单独地介绍它们，但是要看看这两个结构如何与刚才所描述的主要结构相联系。

在 VFS 中有一些与操作相关的结构，这些结构所表示的对象就是操作所施加的实体。在每个对象的操作表中都定义有这些操作。操作表实际上就是函数指针的列表。描述它们的时候，我们为每个对象定义操作表。现在，我们具体分析一下这些结构体。（注意，为了简短、清晰起见，我们没有考虑任何加锁机制）。

1. 超级块结构

当文件系统被安装时，有关它的所有信息均被存放在 `super_block` 结构体中。每个安装好的文件系统都有一个超级块结构。我们先说明这个结构的定义，然后，对比较重要的一些字段进行解释：

```
-----
include/linux/fs.h
666 struct super_block {
667     struct list_head    s_list;
668     dev_t                s_dev;
669     unsigned long        s_blocksize;
670     unsigned long        s_old_blocksize;
671     unsigned char        s_blocksize_bits;
672     unsigned char        s_dirt;
673     unsigned long long    s_maxbytes;
674     struct file_system_type *s_type;
675     struct super_operations *s_op;
676     struct dquot_operations *dq_op;
677     struct quotactl_ops    *s_qcop;
678     struct export_operations *s_export_op;
679     unsigned long        s_flags;
680     unsigned long        s_magic;
681     struct dentry         *s_root;
682     struct rw_semaphore    s_umount;
683     struct semaphore       s_lock;
684     int                    s_count;
685     int                    s_syncing;
686     int                    s_need_sync_fs;
687     atomic_t               s_active;
688     void                   *s_security;
689
690     struct list_head       s_dirty;
691     struct list_head       s_io;
692     struct hlist_head      s_anon;
693     struct list_head       s_files;
694
695     struct block_device    *s_bdev;
696     struct list_head       s_instances;
697     struct quota_info       s_dquot;
698
699     char                    s_id[32];
700
701     struct kobject          kobj;
702     void                   *s_fs_info;
703     ...
708     struct semaphore       s_vfs_rename_sem;
709 };
-----
```

第 667 行：s_list 字段是 list_head 类型^①的，它是指向双向循环链表中前一个元素和下

① 第 2 章详细地描述了 list_head 数据类型。

一个元素的指针，这个超级块被嵌入双向循环链表中。如同 Linux 内核中的其他结构体一样，`super_block` 结构体也是用一个双向循环链表来维护的。`list_head` 数据类型包含指向其他两个 `list_head` 的指针：前一个超级块对象的 `list_head` 和下一个超级块对象的 `list_head`。（全局变量 `super_blocks` (`fs/super.c`) 指向链表中的第一个元素。）

第 672 行：在基于磁盘的文件系统中，用最初存放在磁盘特定扇区中的信息（从磁盘装入到内存的超级块结构中）来填充超级块结构，因为 VFS 允许对超级块结构中的字段进行修改，所以，超级块结构中的信息可能发现自己与磁盘上的数据不同步。这个字段标识超级块结构已被修改，于是需要和磁盘进行同步。

第 673 行：这个无符号长整型的字段定义文件系统所允许的最大文件大小。

第 674 行：超级块结构体包含通用文件系统的信息。然而，它必须涉及具体文件系统（例如，MSDOS、ext2、MINIX 和 NFS）的信息。结构体 `file_system_type` 存放具体文件系统的信息，内核中配置的每种类型的文件系统都对应一个这种结构体。这个字段指向某个具体文件系统的结构体，VFS 利用它管理从一般请求到具体文件系统操作的交互过程。

图 6-4 给出了 `file_system_type` 和超级块之间的关系。该图说明了 `superblock->s_type` 字段如何指向 `file_systems` 链表中合适的 `file_system_type` 结构体（6.2.2 节说明了 `file_systems` 链表究竟是什么）。

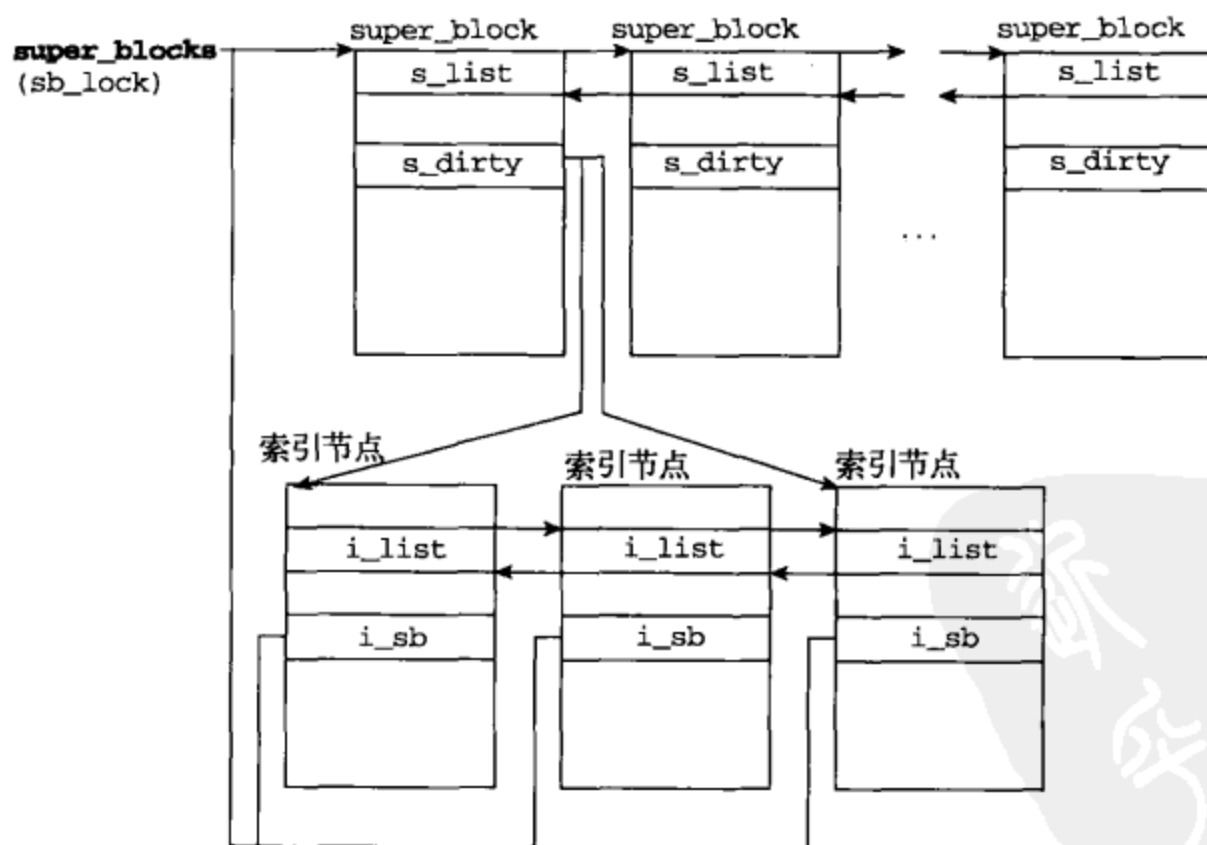


图 6-4 超级块与 `file_system_type` 之间的关系

第 675 行：这个字段是 `super_operation` 结构体类型的指针。这个数据类型存放超级块的操作表。`super_operations` 结构体本身存放一些函数指针，用特定文件系统的超级块操作来初始化这些函数指针。下一节将更详细地解释 `super_operations`。

第 681 行：这个字段是指向目录项结构体的指针。目录项结构体存放文件的路径名。`s_root` 目录项对象的特殊之处在于，它与安装目录相关，属于安装目录的超级块。

第 690 行: `s_dirty` 字段 (不要与 `s_dirt` 混淆) 是一个 `list_head` 结构体, 它指向该文件系统所包含的脏索引节点链表的第一个元素和最后一个元素。

第 693 行: `s_files` 字段是一个 `list_head` 结构体, 它指向文件结构体 (分配给超级块并正在被超级块使用) 链表的第一个元素。在“文件结构”部分, 你会看到它是文件结构所在的三个链表中的一个。

第 696 行: `s_instances` 字段是一个 `list_head` 结构体, 它指向超级块链表中相邻的超级块元素, 该超级块链表中的超级块都是同一种文件系统类型。`file_system_type` 结构体的 `fs_supers` 字段指向 `s_instances` 链表的表头。

第 702 行: 这个 `void *` 数据类型指向超级块的附加信息, 这些附加信息是具体文件系统所特有的 (例如, `ext3_sb_info`)。对于无法抽象出虚拟文件系统超级块概念的具体文件系统来说, 它就像包罗万象的容器, 存放磁盘上超级块中的任何数据。

2. 超级块的操作

超级块的 `s_op` 字段指向文件系统的超级块可以执行的操作表。每个文件系统都有各自的列表, 因为它直接对文件系统的实现进行操作。操作表存放在一个 `super_operations` 类型的结构体中:

```
-----
include/linux/fs.h
struct super_operations {
    struct inode *(*alloc_inode)(struct super_block *sb);
    void (*destroy_inode)(struct inode *);

    void (*read_inode) (struct inode *);

    void (*dirty_inode) (struct inode *);
    void (*write_inode) (struct inode *, int);
    void (*put_inode) (struct inode *);
    void (*drop_inode) (struct inode *);
    void (*delete_inode) (struct inode *);
    void (*put_super) (struct super_block *);
    void (*write_super) (struct super_block *);
    int (*sync_fs)(struct super_block *sb, int wait);
    void (*write_super_lockfs) (struct super_block *);
    void (*unlockfs) (struct super_block *);
    int (*statfs) (struct super_block *, struct kstatfs *);
    int (*remount_fs) (struct super_block *, int *, char *);
    void (*clear_inode) (struct inode *);
    void (*umount_begin) (struct super_block *);

    int (*show_options)(struct seq_file *, struct vfsmount *);
};
-----
```

当对文件系统的超级块初始化时, `s_op` 字段被设置成指向合适的操作表。“从通用文件系统层到特定文件系统层”部分将说明在 `ext2` 文件系统中如何实现这个操作表。表 6-2 给出了超级块

操作的列表。其中某些函数是可选的，只需要用所支持文件系统的操作表子集来填充。对于不支持的特定可选函数，就把操作结构体中的对应字段设置为 NULL。

表6-2 超级块的操作

超级块操作名	说 明
alloc_inode	这是Linux 2.6内核中新增的。它分配并初始化超级块中的vfs索引节点。具体的初始化留给特定的文件系统去完成。在索引节点缓冲上调用kmem_cache_create()或kmem_cache_alloc()（参见第4章）完成真正的分配
destroy_inode	这是Linux 2.6内核中新增的。它回收与超级块相关的具体索引节点。调用kmem_cache_free()完成回收
read_inode	读取inode->i_ino字段所指向的索引节点。用磁盘上的数据更新索引节点中的字段。其中inode->i_op字段尤其重要
dirty_inode	把索引节点放入超级块的脏索引节点链表中。利用superblock->s_dirty字段来引用这个双向循环链表的头和尾。图6-5说明了超级块的脏索引节点链表
write_inode	把索引节点信息写到磁盘上
put_inode	从索引节点缓冲释放索引节点。调用iput()来进行释放
drop_inode	当最后一次访问完索引节点时调用
delete_inode	从磁盘删除索引节点。用于那些不再需要的索引节点。调用generic_delete_inode()完成删除
put_super	释放超级块（例如，当卸载文件系统时）
write_super	把超级块的信息写到磁盘上
sync_fs	目前只能被ext3、Resiserfs、XFS和JFS使用，这个函数把脏的超级块结构体数据写到磁盘上
write_super_lockfs	由ext3、JFS、Resiserfs和XFS使用，这个函数先禁止对文件系统做修改。然后更新磁盘的超级块
unlockfs	解除由write_super_lockfs()加的锁
stat_fs	调用它获得文件系统的统计信息
remount_fs	重新安装文件系统时调用它，用来更新任一安装选项
clear_inode	释放索引节点以及与该索引节点相关的所有页面
umount_begin	在安装操作必须被打断时调用
show_options	用于从已安装的文件系统获得文件系统的信息

我们完成了对超级块结构及其操作的介绍，接下来开始详细地探究索引节点的结构。

3. 索引节点结构

我们说过，索引节点是记录文件信息的结构，例如指向包含文件所有数据的数据块的指针。回想一下，目录、设备以及管道（作为例子）在内核中也被看做文件，因此，索引节点也能表示目录、设备和管道。索引节点对象在文件的整个生命周期中都存在，而且包含在磁盘上维护的数据。

为了便于引用，索引节点用链表来存放。其中一种链表是散列表，它能够减少查找特定索引节点的时间。索引节点也可以在三种双向链表中的一个里面找到自己。表 6-3 给出了这三种链表的类型。图 6-5 显示了超级块结构和它的脏索引节点链表之间的关系。

表6-3 索引节点链表

链 表	i_count	是 否 脏	引用指针
有效未使用的链表	i_count = 0	不脏	inode_unused (全局的)
有效在使用的链表	i_count > 0	不脏	inode_in_use (全局的)
脏索引节点的链表	i_count > 0	脏的	超级块的s_dirty字段

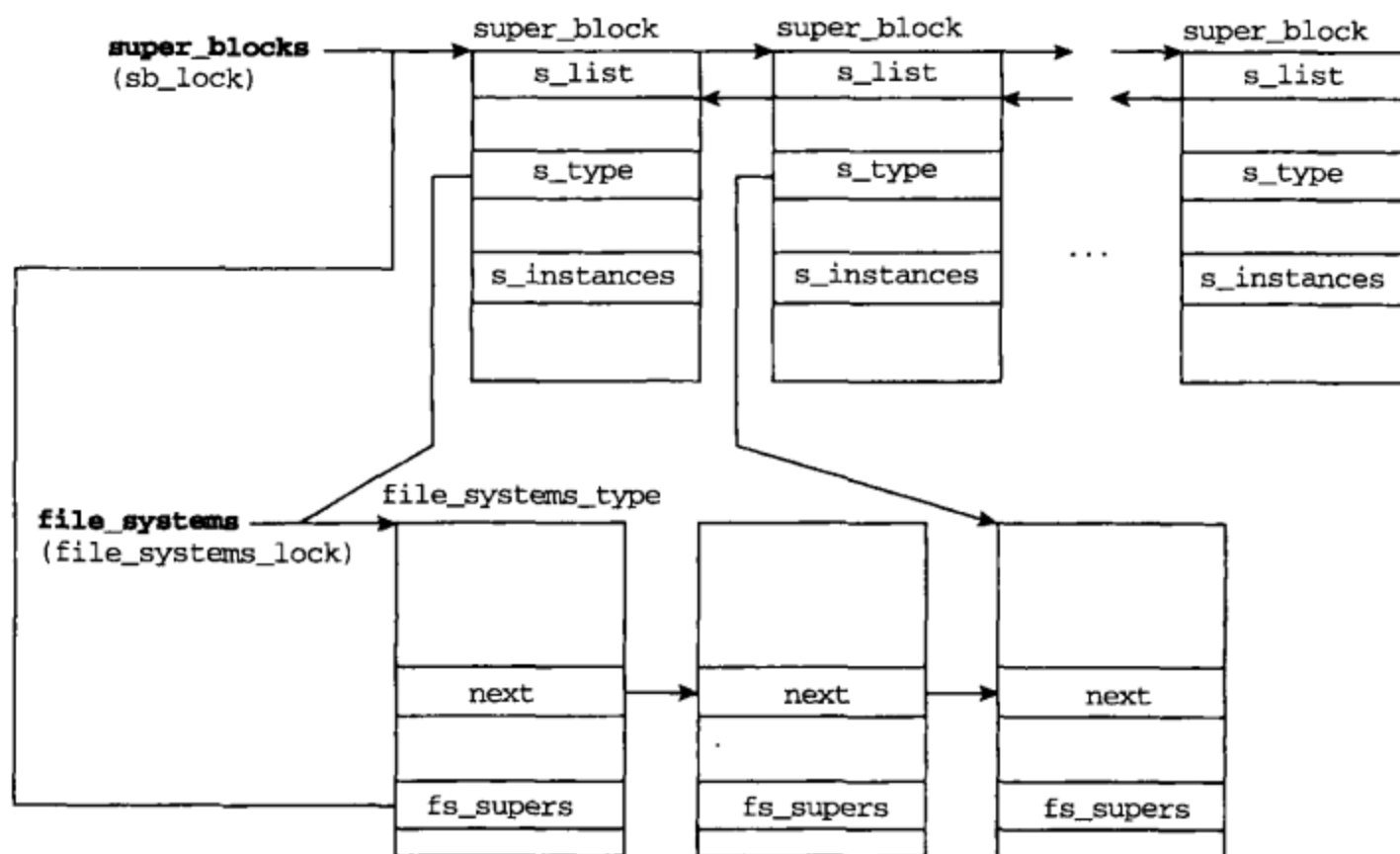


图 6-5 超级块与索引节点之间的关系

索引节点的结构体非常庞大，含有很多字段。下面是对它一小部分字段的描述。

```

-----
include/linux/fs.h
368 struct inode {
369     struct hlist_node  i_hash;
370     struct list_head   i_list;
371     struct list_head   i_dentry;
372     unsigned long      i_ino;
373     atomic_t           i_count;
...
390     struct inode_operations *i_op;
...
392     struct super_block  *i_sb;
...
407     unsigned long      i_state;
...
421 };
-----

```

第 369 行：`i_hash` 字段是 `hlist_node` 类型的^①。它包含一个指向散列表的指针，散列表用来加速索引节点的查找。索引节点的散列表由全局变量 `inode_hashtable` 来引用。

第 370 行：这个字段链接索引节点链表中相邻的结构体。索引节点可以在三个链表中的某一个上找到自己。

第 371 行：这个字段指向文件对应的目录项结构体的链表。目录项结构体中包含该文件所在的路径名，而文件由索引节点表示。如果文件有多个别名，则它可能拥有多个目录项结构体。

第 372 行：这个字段存放唯一的索引节点号。当在一个特定的超级块中分配索引节点时，这个号是根据前一个已分配的索引节点 ID 自动增加的值。当调用超级块操作 `read_inode()` 时，这个字段所表示的索引节点被从磁盘读入内存。

第 373 行：`i_count` 字段是一个计数器，每使用一次索引节点就增加 1。值 0 表示索引节点未被使用，正数表示索引节点正在使用。

第 392 行：这个字段存放的指针指向文件所在文件系统的超级块。图 6-5 说明了超级块的脏索引节点链表中的所有索引节点如何让它们的 `i_sb` 字段指向相同的超级块。

第 407 行：这个字段对应索引节点的状态标志。表 6-4 列出了可能的状态值。

表6-4 索引节点的状态

索引节点的状态标志	说 明
<code>I_DIRTY_SYNC</code>	参见 <code>I_DIRTY</code> 的描述
<code>I_DIRTY_DATASYNC</code>	参见 <code>I_DIRTY</code> 的描述
<code>I_DIRTY_PAGES</code>	参见 <code>I_DIRTY</code> 的描述
<code>I_DIRTY</code>	这个宏关联三个 <code>I_DIRTY_*</code> 标志中的任何一个。它使得能够快速检查这些标志。 <code>I_DIRTY*</code> 标志表示索引节点的内容已被修改，需要进行同步
<code>I_LOCK</code>	对索引节点加锁时设置，解锁时清除。第一次创建索引节点以及涉及 I/O 传送时，索引节点被加锁
<code>I_FREEING</code>	索引节点被删除时设置该标志。因为它将要被删除，这个标志用于把索引节点标记为不可使用，这样就没有人能够再引用它了
<code>I_CLEAR</code>	表示索引节点不再有用
<code>I_NEW</code>	索引节点创建时设置。第一次解锁新索引节点时清除此标志

设置了 `I_LOCK` 或 `I_DIRTY` 标志的索引节点可以在 `inode_in_use` 链表中找到自己。如果这两个标志中的一个没有被设置，索引节点就会被添加到 `inode_unused` 链表中。

4. 目录项结构

目录项结构表示目录的中一项，VFS 用它记录基于目录命名、目录组织以及文件逻辑布局之间的关系。每个目录项对象对应路径名中的一个分量，并且关联与其相关的其他结构和信息。例如，在路径 `(/home/lkp)/Chapter06.txt` 中，包含了 `/`、`home`、`lkp` 这些目录项和 `Chapter06.txt`。每个目录项都涉及分量的索引节点、超级块和相关信息。图 6-6 说明了超级块、索引节点及目录项结构体之间的关系。

^① `hlist_node` 是一个用于双向链表的链表指针类型，与 `list_head` 非常类似。二者之间的差别是链表头 (`hlist_node` 类型) 只包含一个指向第一个元素的指针，而不是两个指针 (第二个指向链表尾)。这减少了散列表的开销。

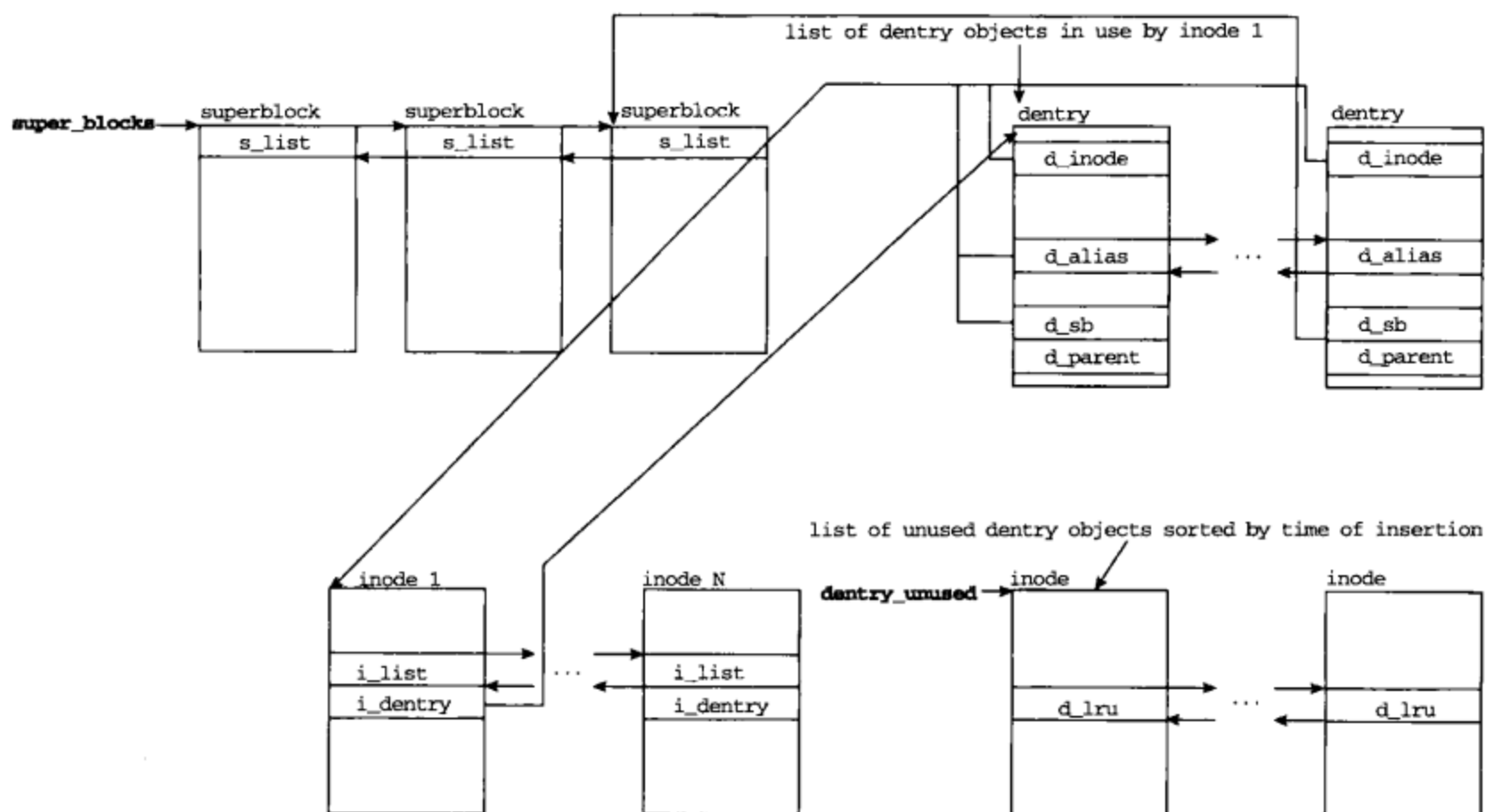


图 6-6 超级块、目录项及索引节点之间的关系

现在来看看 dentry 结构体的部分字段：

```
-----
include/linux/dcache.h
81 struct dentry {
...
85 struct inode * d_inode;
86 struct list_head d_lru;
87 struct list_head d_child; /* child of parent list */
88 struct list_head d_subdirs; /* our children */
89 struct list_head d_alias;
90 unsigned long d_time; /* used by d_revalidate */
91 struct dentry_operations *d_op;
92 struct super_block * d_sb;
...
100 struct dentry * d_parent;
...
105 } ____cacheline_aligned;
-----
```

第 85 行：`d_inode` 字段指向与目录项关联的文件所对应的索引节点。如果目录项对应的路径名分量没有相关索引节点，这个字段的值就是 `NULL`。

第 86~88 行：这里是指向目录项链表中相邻元素的指针。目录项对象能够在表 6-5 所示的其中一种链表上找到自己。

表6-5 目录项链表

链表名	链表指针	说明
已使用的目录项	<code>d_alias</code>	索引节点相同的所有目录项形成一个链表， <code>i_dentry</code> 字段指向该链表的表头
未使用的目录项	<code>d_lru</code>	这些目录项不再使用，但是继续保存，以防路径名中再次使用这个分量

第 91 行: `d_op` 字段指向目录项的操作表。

第 92 行: 这是一个指向超级块的指针, 该超级块就是与目录项所表示的分量相关的超级块。参见图 6-6, 可以了解如何把目录项与超级块结构体关联起来。

第 100 行: 这个字段存放指向父目录项或者路径名中父分量对应的目录项的指针。例如, 在路径名 `/home/paul` 中, 目录项 `paul` 的 `d_parent` 字段指向目录项 `home`, 而 `home` 目录项的 `d_parent` 指向目录项 `/`。

5. 文件结构

VFS 使用的另一个结构是文件结构。当进程对文件进行操作时, 文件结构是 VFS 用于存放进程和文件关联信息的数据类型。与其他结构不同, 文件结构并不存放原始的磁盘数据; 文件结构体在调用 `open()` 系统调用时动态创建, 在调用 `close()` 系统调用时被销毁。回想第 3 章, 在进程的整个生命期中, 由进程所打开的表示文件的文件结构体是通过进程描述符 (`task_struct`) 引用的。图 6-7 说明文件结构体如何与 VFS 的其他 VFS 结构体相关联。 `task_struct` 指向文件描述符表, 该表中存放指向由进程打开的所有文件描述符的指针。回忆一下, 描述符表中的前三项分别对应 `stdin`、`stdout` 和 `stderr` 的文件描述符。

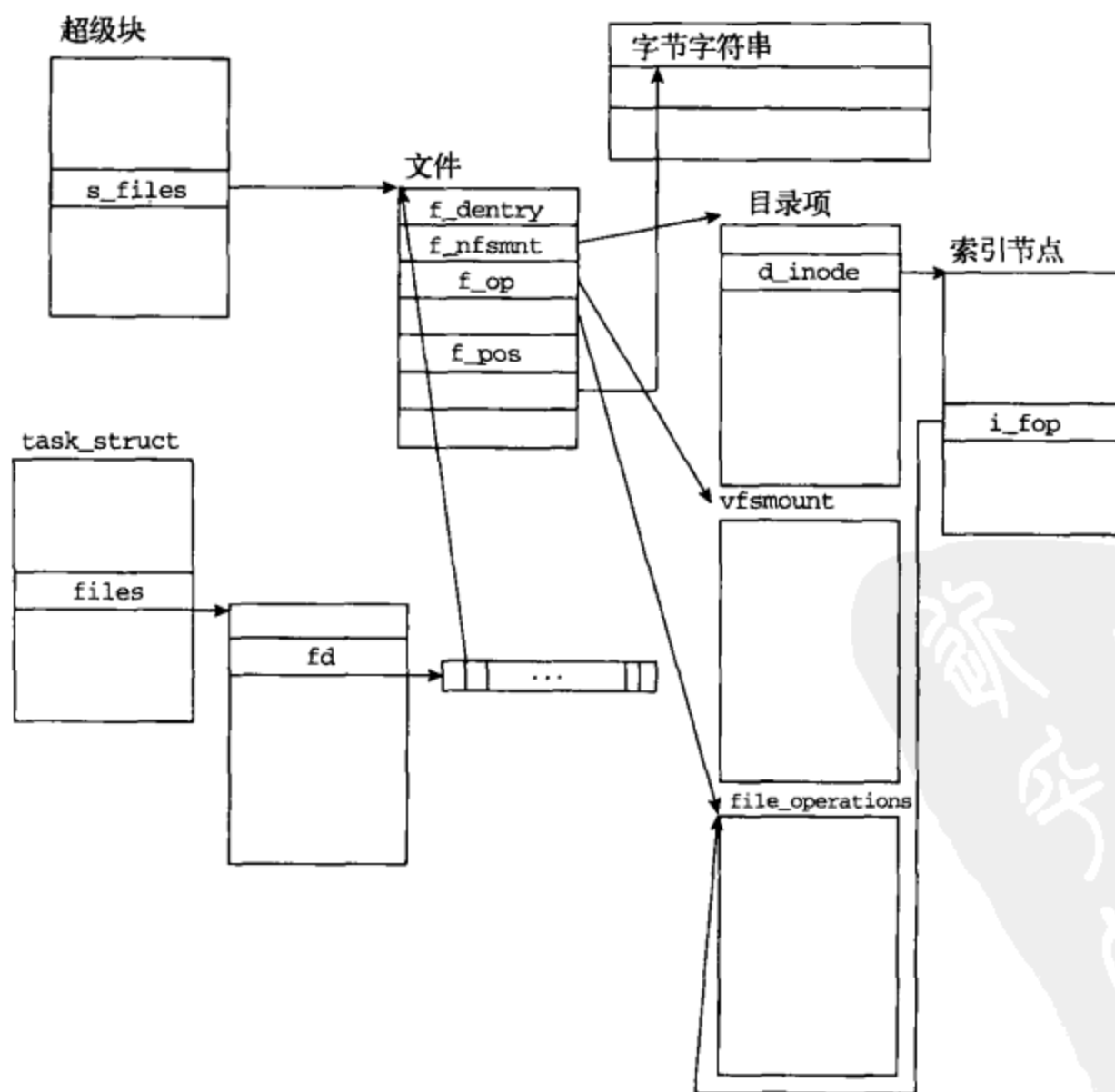


图 6-7 文件对象

内核用双向循环链表来存放文件结构体。根据文件的使用和分配情况, 文件结构体被嵌入三

个链表中的一个。表 6-6 描述了这 3 个链表。

表6-6 文件链表

名 字	指向链表头的指针	说 明
空闲文件对象链表	全局变量 <code>free_list</code>	由所有可用的文件对象组成的双向链表。链表的长度不小于 <code>NR_RESERVED_FILES</code>
正在使用但没有分配的文件对象链表	全局变量 <code>anon_list</code>	由正在被使用但是还没有分配给超级块的所有文件对象组成的双向链表
超级块的文件对象链表	超级块的 <code>s_files</code> 字段	一个双向链表，由包含与超级块相关的文件的所有文件对象组成

内核利用 `get_empty_filp()` 创建文件结构体。该例程返回指向文件结构体的一个指针，如果没有空闲结构体，或是系统已用完了内存就返回 `NULL`。

现在来看看文件结构体中一些比较重要的字段：

```
-----
include/linux/fs.h
506 struct file {
507     struct list_head    f_list;
508     struct dentry        *f_dentry;
509     struct vfsmount      *f_vfsmnt;
510     struct file_operations *f_op;
511     atomic_t             f_count;
512     unsigned int         f_flags;
513     mode_t               f_mode;
514     loff_t               f_pos;
515     struct fown_struct    f_owner;
516     unsigned int         f_uid, f_gid;
517     struct file_ra_state  f_ra;
...
527     struct address_space *f_mapping;
...
529 };
```

第 507 行：`list_head` 类型的 `f_list` 字段存放着指向链表中相邻文件结构体的指针。

第 508 行：这是指向与文件相关的目录项结构体的指针。

第 509 行：这是指向 `vfsmount` 结构体的指针，该结构体与文件所在的已安装文件系统相关。所有已安装的文件系统都有一个用于存放安装的相关信息的 `vfsmount` 结构体。图 6-8 给出了与 `vfsmount` 结构体相关的数据结构。

第 510 行：这是一个指向 `file_operations` 结构体的指针，其中存放的文件操作表用于对文件进行操作。（索引节点的字段 `i_fop` 也指向这个结构体。）图 6-7 说明了这种关系。

第 511 行：多个进程可以并发地访问一个文件。当文件结构体未被使用（因此是可用的）时，`f_count` 字段被设置为 0。当文件结构体与一个文件相关联时，`f_count` 被设置为 1，之后，处理文件的进程每增加一个，`f_count` 就增加 1。因此，如果 4 个不同的进程正在访问同一个文件对象，该文件对象正在使用并且表示一个文件，则文件对象的 `f_count` 字段存放的值就是 5。

第 512 行：`f_flags` 字段包含经由 `open()` 系统调用传入的标志。6.5.1 节将更详细地介绍这个

字段。

第 514 行: `f_pos` 字段存放文件的偏移。这实际上就是文件操作表中某些方法用来指示文件当前位置的读/写指针。

第 516 行: 我们必须知道进程的所有者是谁, 以便操作文件时能够确定文件的访问权限。这些字段分别对应启动进程并打开了文件的用户所拥有的 `uid` 和 `gid`。

第 517 行: 文件可以从页缓存中读取页面, 前面说过, 页缓存是内存中页的集合。预读优化包括预读文件中临近的页面, 在这些页面被请求读入内存之前预先读取它们, 可以减少费时的磁盘访问的次数。`f_ra` 字段存放一个 `file_ra_state` 类型的结构体, 它包含了与文件预读状态相关的所有信息。

第 527 行: 这个字段指向 `address_space` 结构体——对应于该文件所使用的页缓存机制。这在 6.4 节中会更详细地讨论。

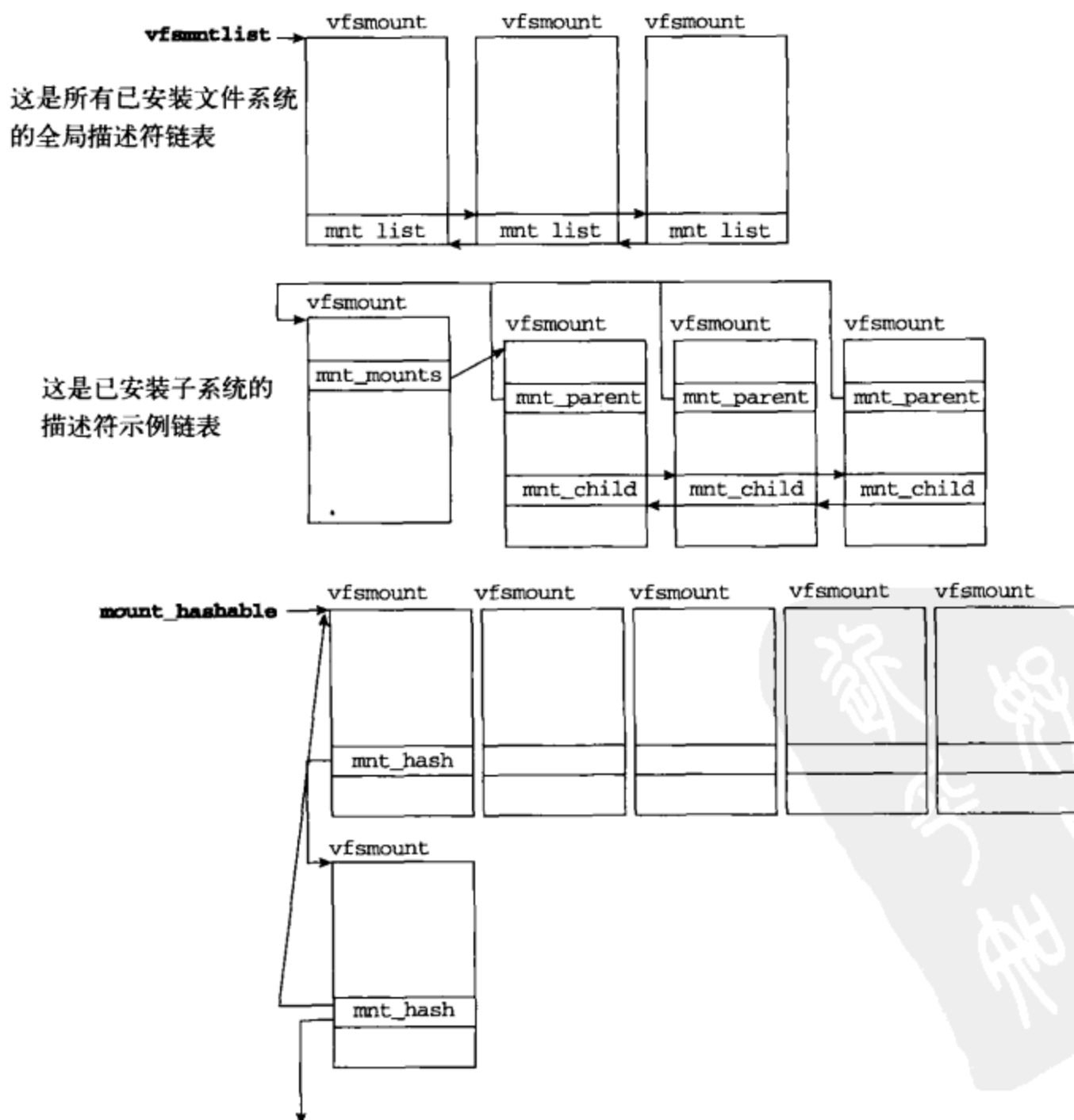


图 6-8 `vfsmount` 对象

6.2.2 全局链表和局部链表的引用

Linux 内核使用全局变量来存放前面提到的指向结构体链表的指针。所有结构体都用双向链表存放。内核存放指向链表头的指针，把它作为链表的访问点。这些结构体都包含 `list_head` 类型的字段^①，用来指向链表中前一个元素和下一个元素。表 6-7 概括了内核存放的全局变量以及这些变量指向的链表的类型。

表6-7 与VFS相关的全局变量

全局变量	结构类型
<code>super_blocks</code>	<code>super_block</code>
<code>file_systems</code>	<code>file_system_type</code>
<code>dentry_unused</code>	<code>dentry</code>
<code>vfsmntlist</code>	<code>vfsmount</code>
<code>inode_in_use</code>	<code>inode</code>
<code>inode_unused</code>	<code>inode</code>

`super_block`、`file_system_type`、`dentry` 和 `vfsmount` 结构体都存放在它们自己的链表中。而索引节点却可能是存放在全局的 `inode_in_use` 上或 `inode_unused` 上，或者是存放在它们所对应的超级块的局部链表上。图 6-9 说明了这些结构体是如何相互关联的。

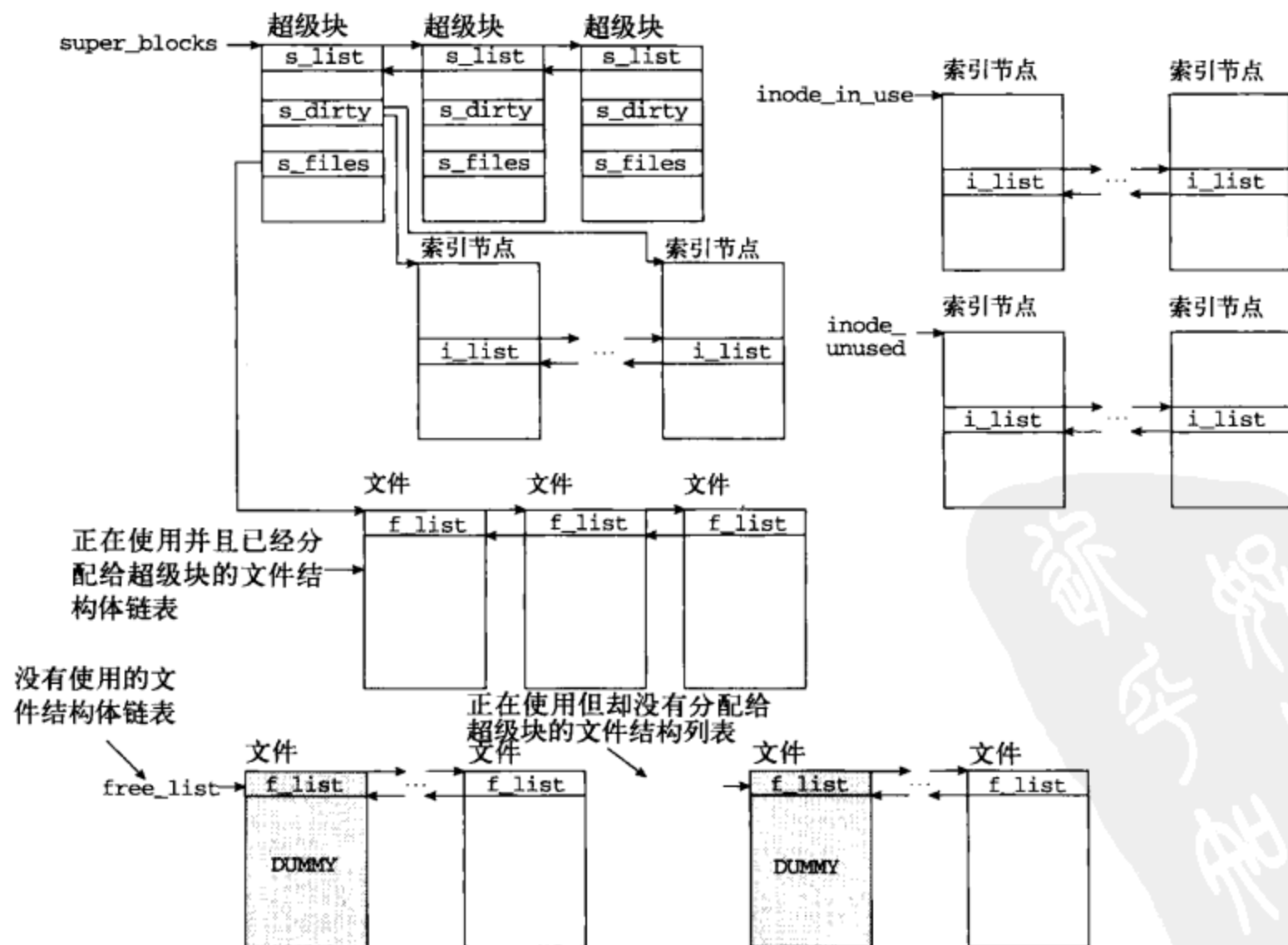


图 6-9 与 VFS 相关的全局变量

① 与我们在“索引节点结构”部分看到的一样，索引节点结构体有一个称为 `hlist_node` 的变体。

`super_blocks` 变量指向超级块链表的头，通过 `s_list` 字段指向链表中前一个元素和下一个元素。超级块结构的 `s_dirty` 字段又依次指向它所拥有的索引节点，这些节点必须与磁盘同步。不在局部的超级块链表上的索引节点就在 `inode_in_use` 链表或 `inode_unused` 链表上。所有索引节点通过 `i_list` 指向索引节点链表中前一个元素和下一个元素。

超级块也通过 `s_files` 指向一个链表的表头，这个链表包含着分配给本超级块的文件结构体。没有分配的文件结构体放到 `anon_list` 链表或 `free_list` 链表中。这两个链表都有一个虚的文件结构体作为链表的表头。所有文件结构体都通过 `f_list` 字段指向其链表中的前一个元素和下一个元素。

参见图 6-6，看看索引节点如何利用 `i_dentry` 指向目录项结构链表。

6.3 与 VFS 相关的结构

除了上述 4 个主要的 VFS 结构之外，还有其他几个结构与 VFS 相互关联：`fs_struct`、`files_struct`、`namespace` 和 `fd_set`。其中，`fs_struct`、`files_struct` 和 `namespace` 结构都是与进程相关的对象，它们包含了与文件相关的数据。图 6-10 说明进程描述符如何与文件相关的结构体相关联。现在，我们来看看这些附加的结构。

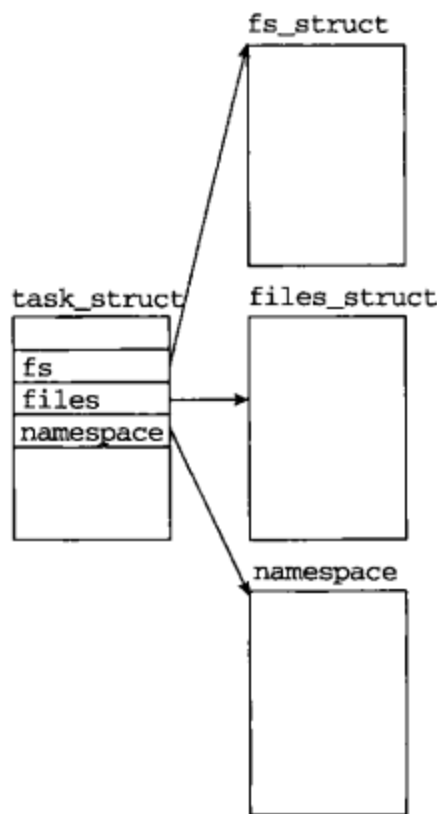


图 6-10 与进程相关的对象

6.3.1 `fs_struct` 结构

在 Linux 中，多个进程能够引用同一个文件。因此，Linux 的 VFS 必须存放有关进程和文件如何交互的信息。例如，在文件操作的权限方面，由某个用户启动的进程和由另一个用户启动的进程是截然不同的。`fs_struct` 结构体中存放将特定进程和文件关联起来的所有信息。在我们考察 `files_struct` 之前必须先考察 `fs_struct` 结构体，因为 `files_struct` 使用了 `fs_struct`

数据类型。

`fs_struct` 可能被多个进程描述符引用,因此,代表文件的 `fs_struct` 由多个 `task_struct` 描述符引用并不稀奇。

```
-----
include/linux/fs_struct.h
7  struct fs_struct {
8      atomic_t count;
9      rwlock_t lock;
10     int umask;
11     struct dentry * root, * pwd, * altpwd;
12     struct vfsmount * rootmnt, * pwdmnt, * altpwrmnt;
13 };
-----
```

1. count

`count` 字段存放引用具体 `fs_struct` 的进程描述符的数目。

2. umask

`umask` 字段用于存放掩码,它表示在已经打开的文件上要设置的访问权限。

3. root、pwd和altpwd

`root` 字段和 `pwd` 字段是两个指针,分别指向与进程的根目录、当前工作目录相关的目录项对象。`Altpwd` 也是一个指针,指向可选根目录的目录项结构。这个字段用于仿真环境。

4. rootmnt、pwdmnt和altpwrmnt

`Rootmnt`、`pwdmnt` 和 `altpwrmnt` 三个指针,分别指向进程根目录、当前工作目录和可选根目录所对应的安装文件系统对象。

6.3.2 files_struct 结构

`files_struct` 包含与打开的文件及其描述符相关的信息。在引言中提到,文件描述符是和打开文件相关的、唯一的整型数据。用内核术语来说,文件描述符就是当前进程的 `task_struct` 的文件对象的 `fd` 数组的下标,也就是 `current->files->fd`。图 6-7 说明了 `task_struct` 的 `fd` 数组,以及它是如何指向文件的文件结构的。

根据共享特性,例如只读或读/写,Linux 可以关联文件描述符的集合。`fd_struct` 结构体表示文件描述符集。`files_struct` 使用这些集合来对它的文件描述符进行分组。

```
-----
include/linux/file.h
22 struct files_struct {
23     atomic_t count;
24     spinlock_t file_lock;
25     int max_fds;
26     int max_fdset;
27     int next_fd;
28     struct file ** fd;
29     fd_set *close_on_exec;
30     fd_set *open_fds;
31     fd_set close_on_exec_init;
-----
```

```

32     fd_set open_fds_init;
33     struct file * fd_array[NR_OPEN_DEFAULT];
34 };

```

第 23 行：与 `fs_struct` 类似，由于 `files_struct` 能够被多个进程描述符引用，所以必须有 `count` 字段。这个字段在内核例程 `fget()` 中增加，而在内核例程 `fput()` 中减小。在文件关闭期间调用这两个函数。

第 25 行：`max_fds` 字段记录进程最多能够打开的文件数。与 `fd_array` 的默认大小 `NR_OPEN_DEFAULT` 有关，`max_fds` 的默认值是 32。当进程想要打开的文件多于 32 个时，就要增加 `max_fds` 的值。

第 26 行：`max_fdset` 字段记录文件描述符的最大数。类似于 `max_fds`，如果进程打开的文件总数超过了这个值，可以增加这个字段的值。

第 27 行：`next_fd` 字段存放下一个将要分配的文件描述符的值。我们看看如何通过文件的打开和关闭来操作它，但是应该清楚一件事：文件描述符是以递增方式分配的，如果先前分配的文件描述符所对应的文件被关闭，那么，`next_fd` 字段就会被设置成被关闭的那个描述符的值。因此，文件描述符是按照最低可用值的方式进行分配的。

第 28 行：`fd` 数组指向打开的文件对象的数组。它默认指向 `fd_array`，`fd_array` 可以存放 32 个描述符。当出现一个大于 32 的文件描述符的请求时，它指向一个新生成的数组。

第 30~32 行：`close_on_exec`、`open_fds`、`close_on_exec_init` 和 `open_fds_init` 都是 `fd_set` 类型的字段。前面已经提到，`fd_set` 结构体用于存放文件描述符集。在对每个字段单独解释之前，我们先来看看 `fd_set` 结构体。

`Fd_set` 数据类型可被追溯到一个存放无符号长整型数组的结构体——数组的每个元素都存放一个文件描述符。

```

-----
include/linux/types.h
22  typedef __kernel_fd_set  fd_set;
-----

```

`fd_set` 数据类型就是 `__kernel_fd_set` 类型。这种数据类型的结构体用于存放无符号长整型数组。

```

-----
include/linux/posix_types.h
36  typedef struct {
37     unsigned long fds_bits [__FDSET_LONGS];
38 } __kernel_fd_set;
-----

```

`__FDSET_LONGS` 在 32 位系统中的值是 32，而在 64 位系统中的值是 16，这确保 `fd_sets` 总是拥有 1024 大小的位图。此处定义了 `__FDSET_LONGS`：

```

-----
include/linux/posix_types.h
6  #undef __NFDBITS

```

```

7  #define __NFDBITS (8 * sizeof(unsigned long))
8
9  #undef __FD_SETSIZE
10 #define __FD_SETSIZE 1024
11
12 #undef __FDSET_LONGS
13 #define __FDSET_LONGS (__FD_SETSIZE/__NFDBITS)
-----

```

可以用 4 个宏来操作这些文件描述符的集合（参见表 6-8）。

表6-8 文件描述符集的宏

宏	说 明
FD_SET	在集合中设置文件描述符
FD_CLR	从集合中清除文件描述符
FD_ZERO	清除文件描述符集
FD_ISSET	如果文件描述符被设置，则返回

现在，我们来看看各个字段。

1. close_on_exec

close_on_exec 字段是指向一个文件描述符集的指针，这些文件描述符标记为“执行 exec()时关闭”。close_on_exec 初始化为（并且通常）指向 close_on_exec_init 字段。如果被标记为“执行 exec()时关闭”的文件描述符数超出 close_on_exec_init 位字段大小时，则修改 close_on_exec 字段。

2. open_fds

open_fds 字段是指向文件描述符集的指针，这些文件描述符被标记为“打开”。和 close_on_exec 类似，它最开始指向 open_fds_init，并且如果被标记为“打开”的文件描述符数超出 open_fds_init 位字段的大小时，则修改 open_fds 字段。

3. close_on_exec_init

close_on_exec_init 字段存放位字段，用于记录在执行 exec()时要关闭的文件的文件描述符。

4. open_fds_init

open_fds_init 字段存放位字段，用于记录被打开的文件对应的文件描述符。

5. fd_array

fd_array 指针数组指向前 32 个打开的文件描述符。

下面通过宏 INIT_FILES 来初始化 fs_struct 结构体：

```

-----
include/linux/init_task.h
6  #define INIT_FILES \
7  {
8  .count      = ATOMIC_INIT(1),
9  .file_lock  = SPIN_LOCK_UNLOCKED,

```

```

10 .max_fds    = NR_OPEN_DEFAULT,
11 .max_fdset  = __FD_SETSIZE,
12 .next_fd    = 0,
13 .fd         = &init_files.fd_array[0],
14 .close_on_exec = &init_files.close_on_exec_init,
15 .open_fds    = &init_files.open_fds_init,
16 .close_on_exec_init = { { 0, } },
17 .open_fds_init = { { 0, } },
18 .fd_array    = { NULL, }
19 }

```

图 6-11 给出了初始化后的 fs_struct。

```

include/linux/file.h
6  #define NR_OPEN_DEFAULT BITS_PER_LONG

```

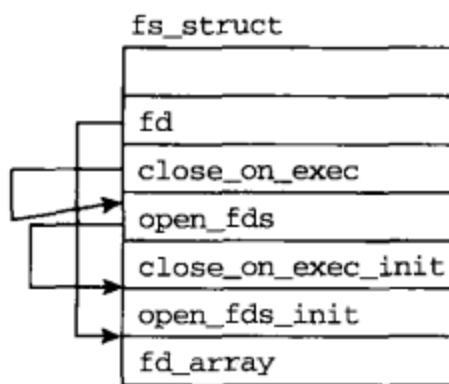


图 6-11 初始化后的 fs_struct

NR_OPEN_DEFAULT 的全局定义被设置为 BITS_PER_LONG。BITS_PER_LONG 在 32 位的系统中是 32，在 64 位的系统中是 64。

6.4 页缓存

在引言中，我们提到页缓存（Page Cache）是内存中页的集合。当频繁访问数据时，能够快速访问数据是至关重要的。当在两个设备之间复制和同步数据时，其中的一个设备通常在存储量方面小于另一个设备，但是却允许更快地访问，我们把这个存储量小但速度快的设备叫做缓存（Cache）。页缓存就是操作系统在内存中存放部分硬盘数据以加快访问的一种方式。现在，我们来看看其工作和实现机理。

对硬盘上的文件执行写操作时，那个文件被分割成称为页的块，这些块被换入内存（RAM）。操作系统修改内存中的页，随后，更新的页被写入磁盘。

如果页从硬盘复制到 RAM（这被称为换入内存），它可能是干净的，也可能变脏。脏页在内存中被修改，但是这个修改还没有写到磁盘上。干净的页在内存中的状态和在磁盘上的状态完全一致。

在 Linux 中，内存被分成区^①。每个区拥有活跃页的链表和非活跃页的链表。如果某个页面已经有一段时间处于不活跃状态，它就会被换出（写回磁盘），从而释放内存。区链表中的每个页均有一个指向 `address_space` 结构体的指针。每个 `address_space` 有一个指向 `address_space_operations` 结构体的指针。通过调用 `address_space_operation` 结构体的 `set_dirty_page()` 函数把页标记为脏。图 6-12 说明了这种依赖关系。

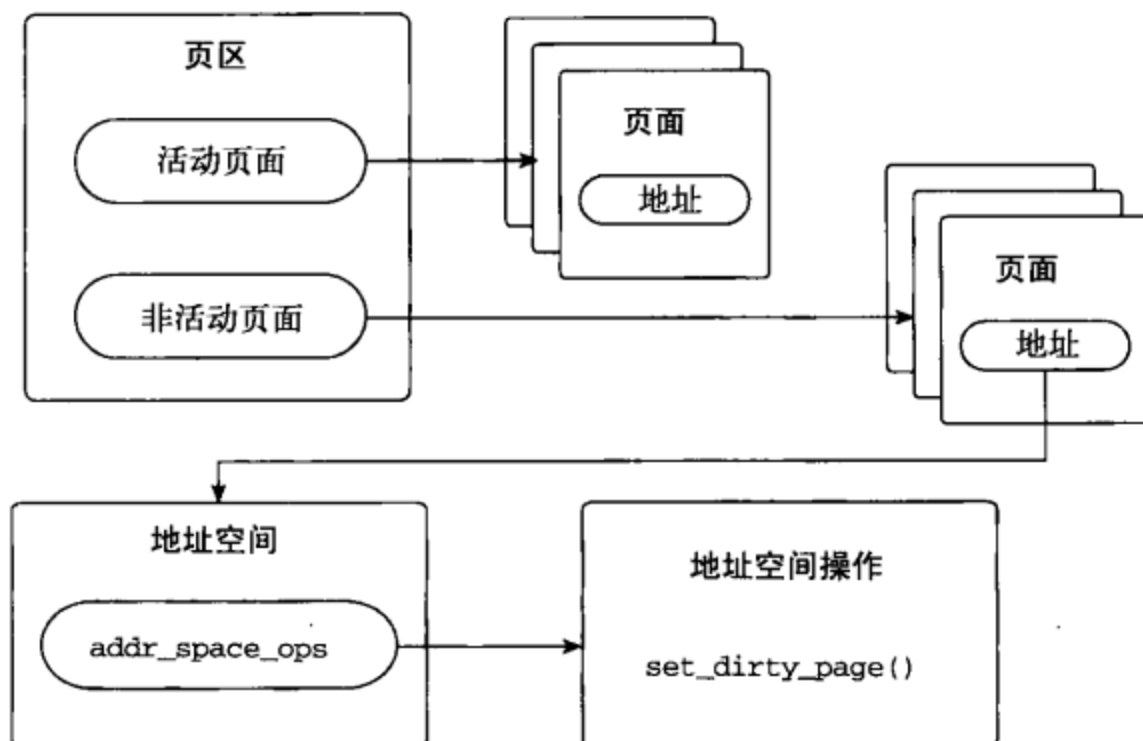


图 6-12 页缓存和区

6.4.1 address_space 结构

页缓存的核心是 `address_space` 对象。让我们来详细分析这个对象：

```

-----
include/linux/fs.h
326 struct address_space {
327     struct inode *host; /* owner: inode, block_device */
328     struct radix_tree_root page_tree; /* radix tree of all pages */
329     spinlock_t tree_lock; /* and spinlock protecting it */
330     unsigned long nrpages; /* number of total pages */
331     pgoff_t writeback_index; /* writeback starts here */
332     struct address_space_operations *a_ops; /* methods */
333     struct prio_tree_root i_mmap; /* tree of private mappings */
334     unsigned int i_mmap_writable; /* count VM_SHARED mappings */
335     struct list_head i_mmap_nonlinear; /* list VM_NONLINEAR mappings */
336     spinlock_t i_mmap_lock; /* protect tree, count, list */
337     atomic_t truncate_count; /* Cover race condition with truncate */
338     unsigned long flags; /* error bits/gfp mask */
339     struct backing_dev_info *backing_dev_info; /* device readahead, etc */
340     spinlock_t private_lock; /* for use by the address_space */
  
```

① 关于内存区的更多信息参见第 4 章。

```

341 struct list_head private_list; /* ditto */
342 struct address_space *assoc_mapping; /* ditto */
343 };

```

结构体中内联的注释已经描述得相当清楚。一些附加的说明也许有助于理解如何操作页缓存。

通常, `address_space` 关联索引节点以及指向这个索引节点的 `host` 字段。不过, 一般来说, 页缓存和地址空间结构并不需要这个字段。如果 `address_space` 对应的内核对象不是索引节点, 那么它可能为空。

`address_space` 结构体有一个直观上让人非常熟悉的字段: `address_space_operations`。与文件结构的 `file_operations` 非常类似, `address_space_operations` 包含 `address_space` 的有效操作的相关信息。

```

-----
include/linux/fs.h
297 struct address_space_operations {
298     int (*writepage)(struct page *page, struct writeback_control *wbc);
299     int (*readpage)(struct file *, struct page *);
300     int (*sync_page)(struct page *);
301
302     /* Write back some dirty pages from this mapping. */
303     int (*writepages)(struct address_space *, struct writeback_control *);
304
305     /* Set a page dirty */
306     int (*set_page_dirty)(struct page *page);
307
308     int (*readpages)(struct file *filp, struct address_space *mapping,
309                     struct list_head *pages, unsigned nr_pages);
310
311     /*
312     * ext3 requires that a successful prepare_write() call be followed
313     * by a commit_write() call - they must be balanced
314     */
315     int (*prepare_write)(struct file *, struct page *, unsigned, unsigned);
316     int (*commit_write)(struct file *, struct page *, unsigned, unsigned);
317     /* Unfortunately this kludge is needed for FIBMAP. Don't use it */
318     sector_t (*bmap)(struct address_space *, sector_t);
319     int (*invalidatepage)(struct page *, unsigned long);
320     int (*releasepage)(struct page *, int);
321     ssize_t (*direct_IO)(int, struct kiocb *, const struct iovec *iov,
322                          loff_t offset, unsigned long nr_segs);
323 };
-----

```

这些函数的意义不言自明。`readpage()`和 `writepage()`分别读写与地址空间相关的页。利用 `readpages()`和 `writepages()`可以读写多个页。日志文件系统(例如 `ext3`)可以提供 `prepare_write()`和 `commit_write()`函数。

当内核在页缓存中查找一个页时, 它必须快速查找。同样地, 每个地址空间有一个 `radix_tree`, 它完成一个快速的搜索, 以确定查找的页是否在页缓存中。

图 6-13 说明了文件、索引节点、地址空间以及页之间是如何互相关联的；这副图对于即将进行的页缓存代码的分析很有用处。

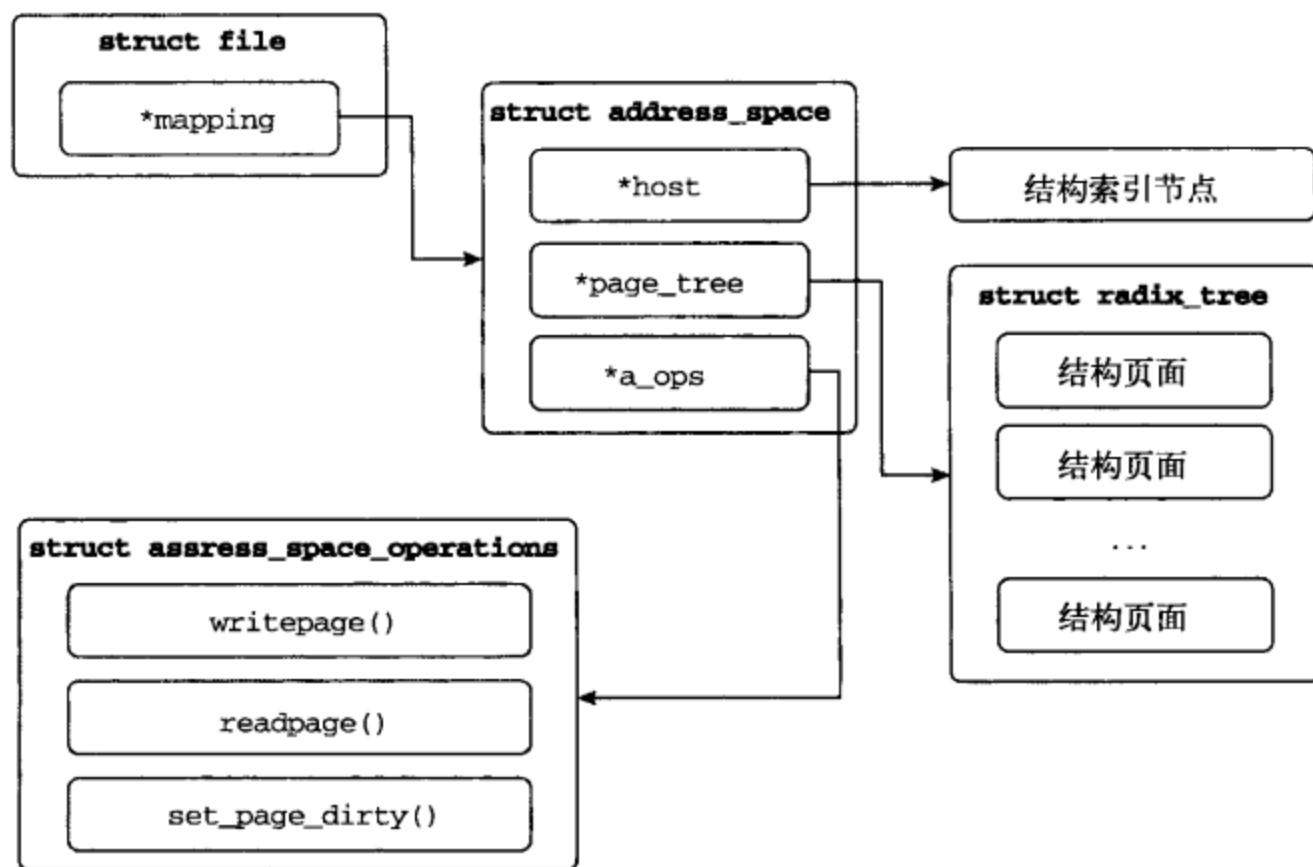


图 6-13 文件、索引节点、地址空间和页

6.4.2 buffer_head 结构

Linux 内核把块设备上的每个扇区表示为 **buffer_head** 结构体。**buffer_head** 包含把物理扇区映射成物理内存中一个缓冲区所有必要的信息。图 6-14 中说明了 **buffer_head** 结构体。

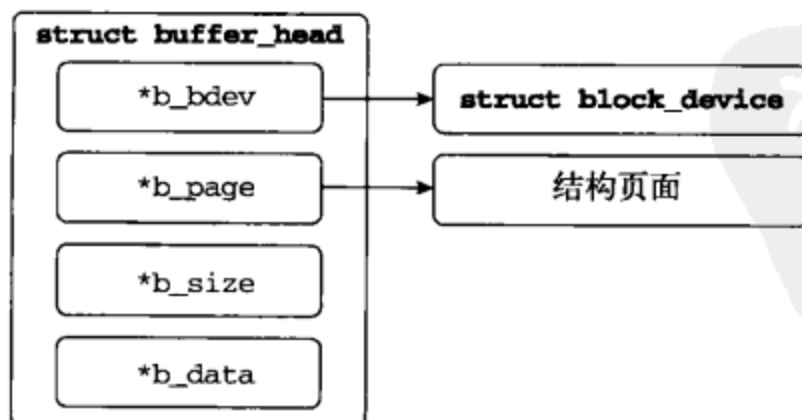


图 6-14 buffer_head 结构体

```

-----
include/linux/buffer_head.h
47 struct buffer_head {
48     /* First cache line: */
49     unsigned long b_state; /* buffer state bitmap (see above) */
50     atomic_t b_count; /* users using this block */

```

```

51 struct buffer_head *b_this_page; /* circular list of page's buffers */
52 struct page *b_page; /* the page this bh is mapped to */
53
54 sector_t b_blocknr; /* block number */
55 u32 b_size; /* block size */
56 char *b_data; /* pointer to data block */
57
58 struct block_device *b_bdev;
59 bh_end_io_t *b_end_io; /* I/O completion */
60 void *b_private; /* reserved for b_end_io */
61 struct list_head b_assoc_buffers; /* associated with another mapping */
62 };

```

buffer_head 结构体所涉及的物理扇区是设备 b_dev 上的逻辑块 b_blocknr。

buffer_head 结构体所涉及的物理内存是起始地址为 b_data 而块大小为 b_size 字节的内存块。这个内存块在物理页 b_page 中。

buffer_head 结构体中的其他定义用于管理物理扇区如何映射到物理内存这样的事务。(因为这牵扯到了 bio 结构体，而不仅仅是 buffer_head，所以，有关结构体 buffer_head 更详细的信息请参见 mpage.c)。

第4章中提到，Linux 内核中每个物理内存页由页结构体表示。页由许多 I/O 块组成。因为每个 I/O 块可能没有页那么大（尽管块可能更小），所以页是由一个或多个 I/O 块组成的。

在较早的 Linux 版本中，块的 I/O 只能利用缓冲区来完成，但在 Linux 2.6 中，一个新方法被开发出来，那就是利用 bio 结构。这个新方法允许 Linux 内核用一种更容易管理的方式把块 I/O 组织在一起。

假设我们要写一个文本文件的开始部分和结束部分。这个更改可能需要两个 buffer_head 结构体用于数据传送：一个指向开始部分，一个指向结束部分。bio 结构允许文件操作仅用一个结构体把不连续的块集合到一起。这种查看缓冲和页的新方法发生在查看缓冲区 (buffer) 的连续内存段时。bio_vec 结构体表示缓冲区中的连续内存段。bio_vec 结构体如图 6-15 所示。

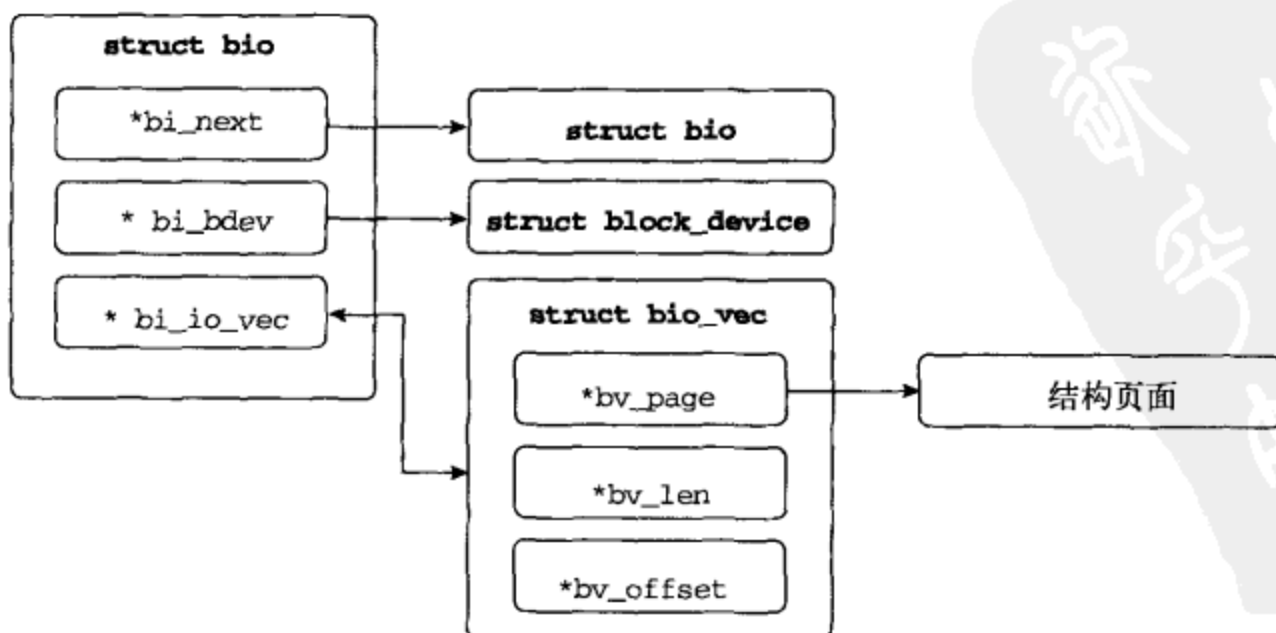


图 6-15 bio 结构

```

-----
include/linux/bio.h
47 struct bio_vec {
48     struct page *bv_page;
49     unsigned int bv_len;
50     unsigned int bv_offset;
51 };
-----

```

bio_vec 结构体存放的信息包括：一个指向页的指针、内存段的长度、段在页中的偏移量。

bio 结构由一个 bio_vec 结构体数组组成（连同其他的管理字段）。因此，bio 结构表示一个或多个缓冲区的多个连续的内存段，这些缓冲区又属于一个或多个页^①。

6.5 VFS 的系统调用和文件系统层

至此，我们介绍了与 VFS 相关的所有结构以及页缓存。接下来，我们将集中精力研究用于文件操作的两个系统调用，并追踪它们的执行直到内核层面。我们来看看 open()、close()、read() 以及 write() 系统调用如何利用先前所描述的那些结构。

之前已经提到，在 VFS 中，文件被视为纯抽象的概念。你能够打开、读、写、关闭一个文件，但是实际发生的细节对 VFS 层并不重要。第 5 章介绍了这些细节。

与 VFS 对应的是文件系统特有的层，这个层把 VFS 的文件 I/O 转换成页和块。因为在计算机系统中你可以拥有许多具体的文件系统类型，例如 ext2 格式的硬盘和 iso9660 的 cdrom，所以，文件系统层可被分成两个主要部分：通用文件系统的操作和具体文件系统的操作（参见图 6-3）。

依照自上而下的方法，本节追踪从 VFS 调用 read() 或 write() 而发出的读写请求着手，穿过文件系统层，一直到具体的块 I/O 请求被提交给块设备驱动程序为止。在这个过程中，我们从通用文件系统层转移到特定文件系统层。在此，以 ext2 文件系统驱动程序作为特定文件系统层的一个实例，但是要切记：访问哪个文件系统的驱动程序取决于正在操作的文件。当我们进行研究时，也会遇到页缓存，它是位于 Linux 通用文件系统层的一种结构。在较早的 Linux 版本中，有缓冲区缓存和页缓存，但在 2.6 内核中，页缓存代替了缓冲区缓存的所有功能。

6.5.1 open()

当进程想要读写文件内容时，它就发出 open() 系统调用：

```

-----
synopsis
#include <sys/types.h>
#include <sys/stat.h>
#include <fcntl.h>
int open(const char *pathname, int flags);
int open(const char *pathname, int flags, mode_t mode);
int creat(const char *pathname, mode_t mode);
-----

```

open 系统调用的参数包括：文件的路径名、标识所打开文件访问模式的标志和权限的位掩码

^① 关于 bio 结构的更多信息参见 include/linux/bio.h。

(文件是否被创建)。open()返回所打开的文件的文件描述符(如果打开成功)或错误码(如果打开失败)。

flags 参数是通过按位或一个或者多个常量来形成的,这些常量在 include/linux/fcntl.h 中定义。表 6-9 列出了 open()使用的标志,以及相应的常量值。O_RDONLY、O_WRONLY 或 O_RDWR 标志只能指定其中一种。其他的标志是可选的。

表6-9 open()的标志

标志名	值	说明
O_RDONLY	0	以只读方式打开文件
O_WRONLY	1	以只写方式打开文件
O_RDWR	2	以读写方式打开文件
O_CREAT	100	表示如果文件不存在,则创建该文件。设置了此标志的open()函数等同于creat()函数
O_EXCL	200	与O_CREAT一起使用,这个标志表示如果文件不存在,则open()失败
O_NOCTTY	400	在路径名引用终端设备的情况中,进程不应该把它当做控制终端
O_TRUNC	0x1000	如果文件存在,把它截为空
O_APPEND	0x2000	从文件末开始写
O_NONBLOCK	0x4000	以非阻塞模式打开文件
O_NDELAY	0x4000	和O_NONBLOCK的值相同
O_SYNC	0x10000	文件的写操作必须等待物理I/O的完成。应用于块设备上的文件
O_DIRECT	0x20000	对文件操作,让I/O上的高速缓存达到最小化
O_LARGEFILE	0x100000	大文件系统允许的文件长度大于31位表示的文件长度。这个标志确保可以打开大文件
O_DIRECTORY	0x200000	如果路径名表示的不是目录,则打开失败
O_NOFOLLOW	0x400000	如果路径名是符号链,则打开失败

让我们来看看系统调用:

```
-----
fs/open.c
927 asmlinkage long sys_open (const char __user * filename, int flags, int mode)
928 {
929     char * tmp;
930     int fd, error;
931
932     #if BITS_PER_LONG != 32
933     flags |= O_LARGEFILE;
934     #endif
935     tmp = getname(filename);
936     fd = PTR_ERR(tmp);
937     if (!IS_ERR(tmp)) {
938         fd = get_unused_fd();
939         if (fd >= 0) {
940             struct file *f = filp_open(tmp, flags, mode);
941             error = PTR_ERR(f);
942             if (IS_ERR(f))
```

```

943     goto out_error;
944     fd_install(fd, f);
945 }
946 out:
947     putname(tmp);
948 }
949     return fd;
950
951 out_error:
952     put_unused_fd(fd);
953     fd = error;
954     goto out;
955 }

```

第 932~934 行：检查我们的系统是否是非 32 位的。如果不是 32 位的，就启用大文件系统的支持标志 `O_LARGEFILE`。这使得 `sys_open` 函数能够打开的文件，其长度大于 31 位表示的文件长度。

第 935 行：`getname()` 例程通过调用 `strncpy_from_user()` 把文件名从用户空间复制到内核空间。

第 938 行：`get_unused_fd()` 例程返回第一个可用的文件描述符（或者说 `fd` 数组的索引：`current->files->fd`），并把它标记为忙。局部变量 `fd` 被设置成这个值。

第 940 行：`filp_open()` 函数完成 `open` 系统调用的大部分工作，并且返回关联进程和文件的文件结构体。接下来详细分析 `filp_open()` 例程：

```

-----
fs/open.c
740 struct file *filp_open(const char * filename, int flags, int mode)
741 {
742     int namei_flags, error;
743     struct nameidata nd;
744
745     namei_flags = flags;
746     if ((namei_flags+1) & O_ACCMODE)
747         namei_flags++;
748     if (namei_flags & O_TRUNC)
749         namei_flags |= 2;
750
751     error = open_namei(filename, namei_flags, mode, &nd);
752     if (!error)
753         return dentry_open(nd.dentry, nd.mnt, flags);
754
755     return ERR_PTR(error);
}
-----

```

第 745~749 行：这是路径名查找函数，例如 `open_namei()`，其访问模式标志的编码不同于 `open` 系统调用，具有其特有的格式。这几行将访问模式标志复制给 `namei_flags` 变量，并将该标志格式化为可以被 `open_namei()` 解释的访问模式标志。

路径名查找的主要差异可能在于：访问模式也许并没有要求读写权限。但是当试图打开文件

时,这种“无权限”的访问模式没有任何意义,因此 open 系统调用的标志不包括“无权限”。“无权限”用值 00 来表示。“读权限”是通过把低位的值设置为 1 来表示的,而“写权限”是通过把高位的值设置为 1 来表示的。在 include/asm/fcntl.h 中,open 系统调用的标志 O_RDONLY、O_WRONLY 和 O_RDWR 对应的值分别是 00、01 和 02。

namei_flags 变量能够通过把自己和 O_ACCMODE 常量进行逻辑按位与来获得访问模式。O_ACCMODE 的值是 3,如果要和 O_ACCMODE 进行按位与的变量值是 1、2 或 3,计算的结果就为真。因此,如果把 open 系统调用的标志设置成 O_RDONLY、O_WRONLY 和 O_RDWR,给这个值加 1 就可以把它转换成路径名查找格式,这样,当它和 O_ACCMODE “与”的时候,结果就为真。第二次检查仅仅是为了确保当把 open 系统调用标志设置为允许文件截断时,就在访问模式中把高位设置为 1,以表明允许写操作。

第 751 行: open_namei() 例程完成路径名的查找,生成相关的 nameidata 结构体并取得相应的索引节点。

第 753 行: dentry_open() 是围绕 dentry_open_it() 的封装例程, dentry_open_it() 创建并初始化文件结构体。创建文件结构体是通过调用内核例程 get_empty_filp() 实现的。如果 files_stat.nr_files 大于或等于 files_stat.max_files, 该例程就返回 ENFILE, 这种情形表示打开的文件总数已达到系统的最大限制。

接下来分析 dentry_open_it() 例程:

```
-----
fs/open.c
844 struct file *dentry_open_it(struct dentry *dentry, struct 845 vfsmount *mnt, int
    flags, struct lookup_intent *it)
846 {
847     struct file * f;
848     struct inode *inode;
849     int error;
850
851     error = -ENFILE;
852     f = get_empty_filp();
853     ...
855     f->f_flags = flags;
856     f->f_mode = (flags+1) & O_ACCMODE;
857     f->f_it = it;
858     inode = dentry->d_inode;
859     if (f->f_mode & FMODE_WRITE) {
860         error = get_write_access(inode);
861         if (error)
862             goto cleanup_file;
863     }
864     ...
866     f->f_dentry = dentry;
867     f->f_vfsmnt = mnt;
868     f->f_pos = 0;
869     f->f_op = fops_get(inode->i_fop);
870     file_move(f, &inode->i_sb->s_files);
871
872     if (f->f_op && f->f_op->open) {
```